ESCOLA SECUNDÁRIA DA BOA NOVA

Curso Profissional de Técnico de Gestão e Programação de Sistemas Informáticos

StickMania – Jogo de Hóquei em Patins 2D

Rafael da Hora Silveira

junho de 2027



ESCOLA SECUNDÁRIA DA BOA NOVA

Curso Profissional de Técnico/a de Desenvolvimento de Software

StickMania – Jogo de Hóquei em Patins 2D

Sob o acompanhamento do Dr. Fernando Filipe

Rafael da Hora Silveira

Projeto realizado em dupla com Vasco Andreia

junho de 2027

Agradecimentos

Agradecemos ao nosso orientador, Dr. Fernando Filipe, e à professora Cristina Paes pelo acompanhamento, pelas sugestões e pela disponibilidade ao longo do projeto. Agradecemos também aos professores do curso, aos colegas que testaram o jogo e deram opiniões, e às nossas famílias pelo apoio.

Resumo

Este relatório descreve o StickMania, um jogo de computador 2D de hóquei em patins desenvolvido em Python com a biblioteca pygame-ce, no âmbito da Prova de Aptidão Profissional. O jogador controla uma equipa contra o computador, com as oito equipas portuguesas, os seus símbolos e equipamentos. O jogo tem física da bola com atrito e ressaltos, colisões entre jogadores, faltas, guarda-redes, uma inteligência artificial com papéis táticos e três níveis de dificuldade, menus completos e uma câmara em perspetiva que mostra o rinque como numa transmissão de televisão, e um modo carreira em que o jogador escolhe um clube e joga ou simula um campeonato contra todas as outras equipas, guardado numa base de dados SQLite, com os planteis reais dos clubes e a tabela dos melhores marcadores. Os sons (apito, tacadas, público) são criados por síntese no próprio código. O desenvolvimento foi iterativo, com versões jogáveis sucessivas e controlo de versões no GitHub. O relatório apresenta a análise de requisitos, a modelação (casos de uso, arquitetura, classes, estados, fluxogramas e modelo de dados), a implementação dos algoritmos principais, os testes realizados e a gestão do projeto.
Este é o relatório individual de Rafael da Hora Silveira. O StickMania foi desenvolvido em dupla com Vasco Andreia: a divisão de tarefas está na secção 5.4 e o capítulo 3 descreve em pormenor as partes feitas por Rafael da Hora Silveira.

Índice

1. Introdução 10
1.1 Enquadramento e contextualização do tema 10
1.2 Motivação e pertinência do projeto 10
1.3 Objetivos (geral e específicos) 10
1.4 Metodologia de trabalho (processo adotado; planeamento) ◆ MADS 11
1.5 Estrutura do relatório 13
2. Análise do sistema 13
2.1 Descrição do problema e da necessidade a resolver ◆ MADS 13
2.2 Intervenientes e contexto de utilização ◆ MADS 13
2.3 Levantamento e análise de requisitos ◆ MADS 14
2.3.1 Técnicas de levantamento utilizadas ◆ MADS 14
2.3.2 Requisitos funcionais (o que o sistema deve fazer) ◆ MADS 14
2.3.3 Requisitos não funcionais (segurança, autonomia, robustez) ◆ MADS 15
2.4 Modelação do sistema ◆ MADS 15
2.4.1 Cenários de utilização / casos de uso ◆ MADS 15
2.4.2 Arquitetura do sistema (diagrama de blocos: sensores → controlo → atuadores) ◆ MADS 17
2.4.3 Modelação comportamental (fluxogramas / máquina de estados) ◆ MADS 19
2.5 Projeto do hardware 22
2.5.1 Componentes: sensores, atuadores e controlador 22
2.5.2 Esquema elétrico e ligações 23
2.5.3 Estrutura mecânica 23
2.6 Prototipagem (protótipo físico e/ou simulação) ◆ MADS 23
3. Implementação 24
3.1 Montagem do hardware 24
3.2 Programação do controlo (lógica e algoritmos) 24
3.2.1 Trabalho feito por Rafael da Hora Silveira 24
3.2.2 Trabalho feito em conjunto 29
3.2.3 Trabalho feito por Vasco Andreia (resumo) 34
3.3 Integração hardware – software 34
3.4 Decisões técnicas e dificuldades ultrapassadas 35
4. Testes e validação 37
4.1 Plano de testes 37
4.2 Ensaios, calibração e medições 42
4.3 Resultados obtidos e ajustes 43

5. Gestão do projeto 44
5.1 Planeamento e cronograma ◆ MADS 44
5.2 Gestão de riscos ◆ MADS 47
5.3 Controlo de versões e trabalho em equipa (Git/GitHub) 47
5.4 Divisão de tarefas 47
6. Conclusão 49
7. Anexos 50

Índice de figuras e tabelas

Figura 1 – Diagrama de casos de uso
Figura 2 – Arquitetura do sistema em blocos (entrada → controlo → saída)
Figura 3 – Diagrama de classes (simplificado)
Figura 4 – Modelo de dados da base de dados SQLite (modo carreira e carreira de jogador)
Figura 5 – Máquina de estados do jogo
Figura 6 – Fluxograma do ciclo principal do jogo
Figura 7 – Fluxograma de decisão da inteligência artificial
Figura 8 – Fluxograma do modo carreira
Figura 9 – Captura de ecrã do StickMania durante uma partida
Figura 10 – Jogo online: o anfitrião à espera do adversário, com o endereço do computador
Figura 11 – Jogo online no computador do convidado
Figura 12 – Versão do browser: o código de 4 letras do jogo online
Figura 13 – Jogo online no browser
Figura 14 – Modo carreira: escolha da equipa e da dificuldade
Figura 15 – Modo carreira: classificação com a forma, próximo jogo com as probabilidades e resultados da jornada
Figura 16 – Modo carreira: resultado de um jogo simulado
Figura 17 – Modo carreira: fim da época, com o campeão e os melhores marcadores
Figura 18 – Estilo arcade: festa de um golo, com o nome de quem marcou
Figura 19 – Diagrama de Gantt do projeto, com a divisão de tarefas (setembro de 2026 a junho de 2027)

Tabela 1 – Descrição do caso de uso «Jogar partida»
Tabela 2 – Requisitos de hardware e software para correr o jogo
Tabela 3 – Controlos do jogo
Tabela 4 – Plano de testes
Tabela 5 – Medição da jogabilidade antes e depois das correções (média por jogo de 3 min, IA contra IA)
Tabela 6 – Bola e guarda-redes antes e depois das correções (média por jogo de 3 min)
Tabela 7 – Modelo de simulação dos jogos: antigo e novo (600 épocas, 16 800 jogos)
Tabela 8 – Remate na primeira pessoa: percentagem de golos no nível normal (força 65 %)
Tabela 9 – Cronograma do projeto
Tabela 10 – Gestão de riscos
Tabela 11 – Divisão de tarefas

Lista de siglas e abreviaturas

2D – Duas dimensões

FPS – Frames (imagens) por segundo

GR – Guarda-redes

HUD – Heads-Up Display – informação mostrada por cima do jogo (marcador, mini-mapa)

IA – Inteligência Artificial

PAP – Prova de Aptidão Profissional

PNG – Portable Network Graphics – formato de imagem

SQL – Structured Query Language – linguagem de bases de dados

UML – Unified Modeling Language – linguagem de diagramas de software

WAV – Waveform Audio File – formato de ficheiro de som

1. Introdução

1.1 Enquadramento e contextualização do tema

O hóquei em patins é uma modalidade com grande tradição em Portugal. A seleção nacional é uma das mais vitoriosas do mundo e clubes como o FC Porto, o Benfica, o Sporting, o OC Barcelos ou a UD Oliveirense enchem pavilhões todas as épocas. Apesar disso, quase não existem videojogos atuais sobre a modalidade: há muitos jogos de futebol e de basquetebol, mas o hóquei em patins ficou esquecido.

O StickMania é um jogo de computador 2D de hóquei em patins, feito em Python com a biblioteca pygame-ce. O jogador controla uma equipa de cinco elementos (quatro jogadores de campo e um guarda-redes) contra uma equipa controlada pelo computador, num rinque visto de lado e de cima, como numa transmissão de televisão. O projeto foi desenvolvido no âmbito da Prova de Aptidão Profissional (PAP) do Curso Profissional de Técnico de Gestão e Programação de Sistemas Informáticos.

1.2 Motivação e pertinência do projeto

A ideia nasceu do gosto pela modalidade e da vontade de ver um jogo com as equipas portuguesas. Ao mesmo tempo, um jogo é um projeto muito completo para aprender programação: obriga a tratar de física (movimento, atrito, colisões e ressaltos), inteligência artificial, interfaces gráficas, animação, organização do código em módulos e testes.

O projeto é pertinente porque junta quase todas as matérias do curso num produto que qualquer pessoa consegue experimentar, e porque ajuda a divulgar uma modalidade com pouca presença nos videojogos.

1.3 Objetivos (geral e específicos)

Objetivo geral: desenvolver um jogo de hóquei em patins 2D jogável, estável e fluido, em que um jogador humano defronta uma equipa controlada pelo computador.

Objetivos específicos:

- Simular a física da bola e dos jogadores (aceleração, atrito do piso, ressaltos nas tabelas e nas balizas).

- Criar uma inteligência artificial com papéis táticos (avançado, médio e defesa) e três níveis de dificuldade.
- Implementar as regras principais: duas partes com tempo, golos, faltas e reposição da bola ao centro.
- Criar menus completos: menu principal, seleção de equipas, definições (controlos e ecrã inteiro), pausa e ecrã final.
- Mostrar as oito equipas com símbolos e equipamentos com as cores de cada clube.
- Desenhar o rinque com uma câmara em perspetiva, bancadas com público e balizas em 3D.
- Criar um modo carreira: um campeonato contra todas as outras equipas, com classificação, guardado numa base de dados SQLite para se poder continuar mais tarde, e com a opção de simular os jogos com resultados realistas.
- Dar som ao jogo (apito, tacadas, tabelas, postes, golos e público), criado por síntese no código.
- Manter o código organizado em módulos e com controlo de versões no GitHub.

1.4 Metodologia de trabalho (processo adotado; planeamento) ◆ MADS

Foi seguido um processo de desenvolvimento iterativo e incremental. Em vez de fazer tudo de uma vez, o jogo foi construído em pequenas versões, cada uma jogável, que foram sendo melhoradas:

- **Versão 1 –** rinque visto de cima, jogadores a mexer e bola com física simples.
- **Versão 2 –** física da bola corrigida, colisões, guarda-redes e golos.
- **Versão 3 –** interface do mockup (marcador, mini-mapa, cartão do jogador) e símbolos dos clubes.
- **Versão 4 –** mira de remate com o rato, bola colada ao stick, níveis de dificuldade e faltas.
- **Versão 5 –** animações de patinagem, balizas desenhadas no código e câmara em perspetiva com bancadas.
- **Versão 6 –** correção das colisões, da bola e do guarda-redes, nova IA tática e modo carreira (campeonato guardado em SQLite).
- **Versão 7 –** menu principal novo com uma engrenagem para as definições, ecrã inteiro e gráficos melhorados (piso, luzes, sombras e linhas suaves).
- **Versão 8 –** sons, simulação de jogos com resultados realistas, força dos clubes também no jogo e ecrãs do modo carreira redesenhados.
- **Versão 9 –** planteis reais (4 jogadores de campo e 1 guarda-redes, com os nomes de cada clube), marcadores de cada golo e tabela dos melhores marcadores.

- **Versão 10 –** estilo arcade (golos com câmara lenta e confetes, impactos com pausa e tremer, textos de banda desenhada), que se liga e desliga nas Definições.
- **Versão 11 –** remate na primeira pessoa (apontar a qualquer ponto da baliza e escolher a força) e MODO TESTE para treinar remates contra o guarda-redes.
- **Versão 12 –** fase de remate em movimento: o jogador continua a patinar a 50 %, os defesas aparecem no ecrã do remate e podem bloquear a bola, e um stick «SUPER/AZEMAD.» com a bola preta.
- **Versão 13 –** ecrã do remate redesenhado como a imagem de referência: bancada com cadeiras e adeptos, marcador eletrónico, placas de publicidade, piso envernizado e o stick da imagem do emblema.
- Versão 14 – defesas do ecrã do remate com o tamanho certo (já não encolhem nem desaparecem ao avançar), sprites HD com as cores do clube e destaque do defesa que tapa a linha.
- Versão 15 – carreira de jogador: crias o teu jogador (nome, número, posição e clube) e nos jogos só o controlas a ele; nota de 0 a 10 em cada jogo, evolução do geral e propostas de clubes mais fortes no fim da época.
- Versão 16 – vida do jogador: forma, moral, confiança do treinador, fama, estudos e dinheiro, com eventos e escolhas em cada jornada, atividades da semana, lesões, banco e diário.
- Versão 17 – menus com o mesmo estilo (seleção de equipas, pausa e fim de jogo com o pavilhão ou o jogo desfocado por trás), transições suaves entre ecrãs e emblemas mais rápidos a carregar.
- Versão 18 – vida do jogador mais rica: pessoas (mãe, pai, melhor amigo e namoro) com relações e interações, 42 eventos com escolhas que têm consequências mais tarde, bens (carta, carro, casa, agente, capitão) e 14 conquistas.
- Versão 19 – otimização (desenho do jogo de 3,85 para 2,35 ms por frame), arranque sem a janela da consola (jogar.vbs) e limpeza dos ficheiros que o jogo não usa.
- Versão 20 – modo online: jogar contra um amigo noutro computador (rede local ou internet); o anfitrião corre o jogo e envia o estado ao convidado 30 vezes por segundo.
- Versão 21 – versão para o browser publicada no GitHub Pages (rafaelsilveira23.github.io/Stickmaniapap), com jogo online por código de 4 letras (WebRTC).

No fim de cada versão o jogo era testado e as falhas encontradas entravam na lista da versão seguinte. Todo o código está guardado no GitHub, o que permite ver a evolução e voltar atrás quando uma alteração não resulta (por exemplo, uma folha de sprites nova que acabou por ser revertida). O planeamento detalhado está no capítulo 5.

1.5 Estrutura do relatório

Depois desta introdução, o capítulo 2 faz a análise do sistema: o problema, os intervenientes, os requisitos e a modelação com diagramas (casos de uso, arquitetura, classes, estados e fluxogramas). O capítulo 3 descreve a implementação, os algoritmos principais e as dificuldades ultrapassadas. O capítulo 4 apresenta os testes realizados e os resultados. O capítulo 5 trata da gestão do projeto (cronograma, riscos, controlo de versões e divisão de tarefas). Por fim, o capítulo 6 apresenta a conclusão e o capítulo 7 os anexos. A secção 3.2 está dividida em três partes: o trabalho feito por Rafael da Hora Silveira, o trabalho feito em conjunto e um resumo do trabalho feito por Vasco Andreia.

2. Análise do sistema

2.1 Descrição do problema e da necessidade a resolver ◆ MADS

Quem gosta de hóquei em patins praticamente não tem jogos de computador da modalidade. Os poucos que existem são antigos, não têm as equipas portuguesas ou não reproduzem bem o jogo: a bola rápida, os ressaltos nas tabelas, o stick e o jogo atrás da baliza.

A necessidade a resolver é ter um jogo simples de instalar, que funcione num computador normal sem internet, com as equipas portuguesas, e que seja divertido tanto para quem conhece a modalidade como para quem nunca a viu. Para isso o jogo tem de ter uma física credível, adversários controlados pelo computador que joguem de forma inteligente e uma apresentação parecida com a televisão.

2.2 Intervenientes e contexto de utilização ◆ MADS

Intervenientes:

- **Jogador:** a pessoa que usa o jogo. Escolhe as equipas e a dificuldade e controla um jogador de cada vez.
- **Computador (IA):** controla a equipa adversária, os colegas do jogador e os dois guarda-redes.
- **Equipa de desenvolvimento:** Rafael da Hora Silveira e Vasco Andreia, responsáveis pela análise, programação, testes e documentação. A divisão de tarefas está na secção 5.4.

- **Orientadores:** Dr. Fernando Filipe e professora Cristina Paes, que acompanharam e validaram o trabalho.
- **Júri da PAP:** avalia o produto final e este relatório.

Contexto de utilização: o jogo corre num computador com Windows, numa janela de 1280×720 píxeis, e usa teclado e rato. Não precisa de internet nem de criar conta. Arranca com um duplo clique no ficheiro jogar.vbs, que procura o Python e instala o pygame-ce se for preciso.

2.3 Levantamento e análise de requisitos ◆ MADS

Os requisitos dizem o que o jogo tem de fazer e em que condições. Foram tirados do anteprojeto do Stick Mania, a partir do problema, dos objetivos e da organização do jogo. Há requisitos funcionais, que descrevem o que o sistema faz, e requisitos não funcionais, ligados à segurança, à autonomia e à robustez.

2.3.1 Técnicas de levantamento utilizadas ◆ MADS

Para chegar a estes requisitos usámos quatro formas.
Leitura do anteprojeto. Lemos o documento e apontámos o que o jogo tem de ter: física, inteligência artificial, menus, torneio e base de dados.
Análise da modalidade. Usámos as regras básicas do hóquei em patins: duas equipas, guarda-redes, bola, stick, golos, tempo e classificação.
Comparação com outros jogos. Há muitos jogos de futebol e de basquetebol, mas quase nenhum de hóquei em patins atual. Isso mostrou o que faltava.
Revisão em grupo. Rafael Hora Silveira e Vasco Andreia reviram a lista com a professora Cristina Paes. Ainda não foram feitos inquéritos nem entrevistas.

2.3.2 Requisitos funcionais (o que o sistema deve fazer) ◆ MADS

Estes requisitos descrevem o que o jogo deve fazer.
01. O jogo deve abrir uma janela e continuar a correr, lendo o teclado.
02. Deve ter menu principal, escolha de equipas, pausa, definições e ecrã de resultado.
03. Deve deixar pausar e continuar sem perder o marcador nem o tempo.
04. Deve contar o tempo, separar as duas partes, detetar o golo e repor a bola no centro.
05. Deve mostrar o nome ou o emblema das duas equipas.
06. Deve mostrar jogadores de campo e guarda-redes, com stick e bola.
07. Deve mudar a animação do jogador: parado, patinar, conduzir, passar, rematar e chocar.
08. Deve mover o jogador ativo em várias direções, com aceleração e um limite de velocidade.
09. Deve deixar trocar o jogador que está a ser controlado.
10. Deve simular o atrito do rinque de madeira.
11. Deve calcular o movimento da bola, os ressaltos, a condução no stick, o passe e o remate.
12. Deve detetar colisões entre jogadores, bola, balizas e limites do rinque.
13. A equipa do computador deve perseguir a bola, defender, passar e rematar.

14. O guarda-redes deve mover-se na baliza para tentar defender.
15. Durante o jogo deve aparecer o marcador, o tempo, a parte e a pausa.
16. Deve haver um mini-mapa com os dez jogadores e a bola, e os atributos do jogador ativo.
17. No torneio deve atualizar vitórias, empates, derrotas e golos.
18. Deve guardar equipas, jogadores, partidas, estatísticas e classificação em SQLite.
19. Deve dar 3 pontos pela vitória, 1 pelo empate e 0 pela derrota, e ordenar a classificação.
20. Deve ter uma carreira de jogador: criar um jogador e, nos jogos, controlar só esse jogador, com uma nota por jogo, evolução e propostas de outros clubes.
21. Na carreira de jogador deve haver uma vida com escolhas (eventos e atividades) que mexem no hóquei (atributos, lesões, banco e propostas) e fora dele (estudos e dinheiro).
22. A vida do jogador deve ter pessoas com quem se relaciona, escolhas com consequências mais tarde, bens e conquistas.
23. Deve deixar jogar online contra outra pessoa noutro computador (na rede local ou pela internet), cada uma com a sua equipa e a ver o mesmo jogo.
24. Deve poder jogar-se no browser, abrindo um link, sem instalar nada, também online contra um amigo.

2.3.3 Requisitos não funcionais (segurança, autonomia, robustez) ◆ MADS

Estes requisitos descrevem a segurança, a autonomia e a robustez do jogo.
01. As consultas à base de dados devem ser parametrizadas, para ninguém enviar comandos pela interface.
02. Os dados do torneio ficam só no computador. Não é preciso criar conta nem enviar dados para fora.
03. O jogo não guarda dados pessoais sensíveis. Guarda equipas, jogadores, partidas e estatísticas.
04. O jogo funciona sem internet e sem servidor.
05. Dá para continuar o torneio mais tarde, no mesmo computador.
06. O jogo não deve fechar sozinho ao pausar, ao marcar um golo ou ao mudar de parte.
07. A bola e os jogadores não devem ficar presos fora do rinque.
08. Se a gravação falhar, o que já estava guardado não deve ser apagado.
09. Não pode haver resultados impossíveis, como golos negativos.
10. O jogo deve andar de forma fluida, perto de 60 imagens por segundo.
11. O marcador, o tempo e o mini-mapa devem estar visíveis durante a partida.
12. O código fica dividido por partes, para cada um mudar a sua parte sem estragar o resto.

2.4 Modelação do sistema ◆ MADS

2.4.1 Cenários de utilização / casos de uso ◆ MADS

O diagrama de casos de uso mostra o que cada ator pode fazer no sistema. O Jogador escolhe as equipas e a dificuldade, consulta as definições (controlos e ecrã inteiro), joga a

partida, pausa e vê o resultado. Durante a partida pode mover o jogador, passar, rematar na primeira pessoa e trocar de jogador. O Computador controla a equipa adversária e os guarda-redes. No modo carreira, o Jogador joga um campeonato completo e consulta a classificação e o calendário entre os jogos. No MODO TESTE treina remates sozinho contra o guarda-redes.

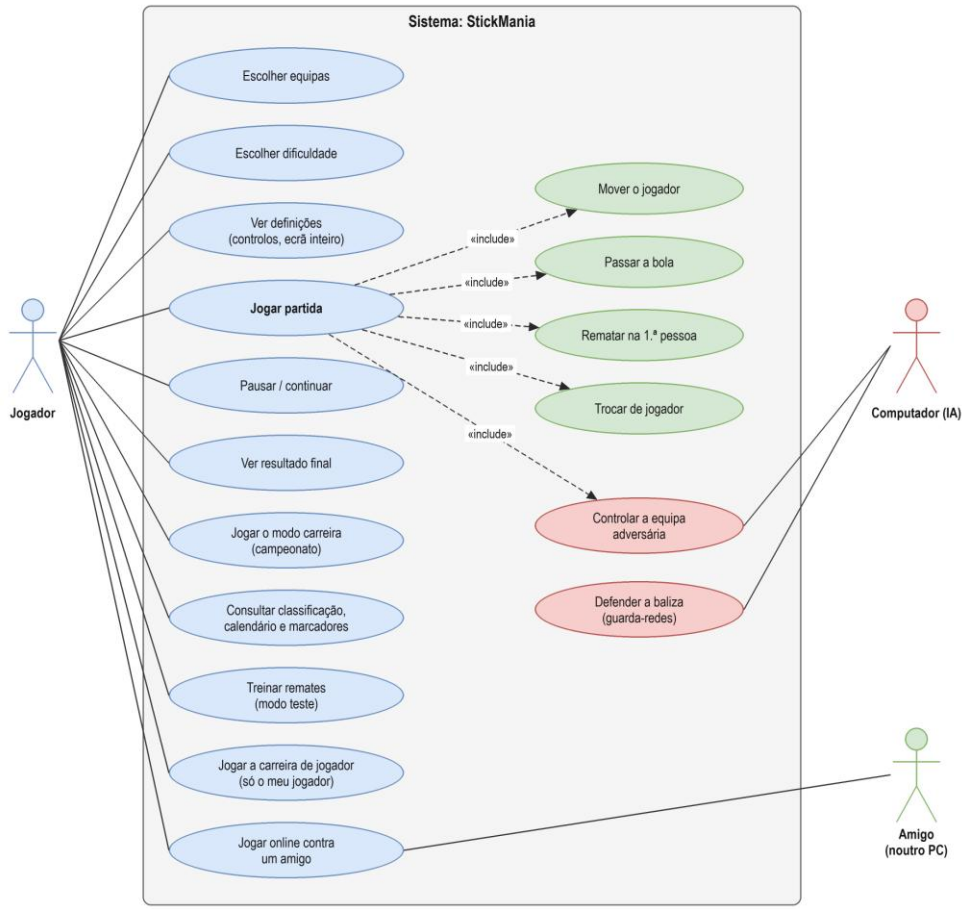


Figura 1 – Diagrama de casos de uso

Tabela 1 – Descrição do caso de uso «Jogar partida»

Campo	Descrição
Ator principal	Jogador (o Computador participa como ator secundário)
Pré-condição	As equipas e a dificuldade foram escolhidas no ecrã de seleção.
Fluxo principal	1. O sistema coloca os jogadores e a bola no centro. 2. O jogador move-se com WASD, passa com Z e remata com X (na primeira pessoa). 3. A IA controla os restantes jogadores. 4. Quando a bola entra na baliza, o golo é somado e a bola volta ao centro. 5. No fim da 1.ª parte há intervalo; no fim da 2.ª parte aparece o resultado final.
Fluxos alternativos	ESC ou P abre a pausa (continuar, reiniciar, definições, menu ou sair). Uma entrada irregular é assinalada como falta e a bola é reposta.
Pós-condição	O resultado final é mostrado; o jogador pode jogar de novo, mudar de equipas ou voltar ao menu.

2.4.2 Arquitetura do sistema (diagrama de blocos: sensores → controlo → atuadores) ◆ MADS

O modelo «sensores → controlo → atuadores» foi adaptado a um sistema de software: os «sensores» são o teclado e o rato, o «controlo» é o ciclo principal do jogo e os «atuadores» são o ecrã e o som. Por baixo ficam os dados que o jogo carrega ao arrancar. Cada bloco corresponde a um ou mais ficheiros Python, o que permite alterar uma parte sem estragar as outras.

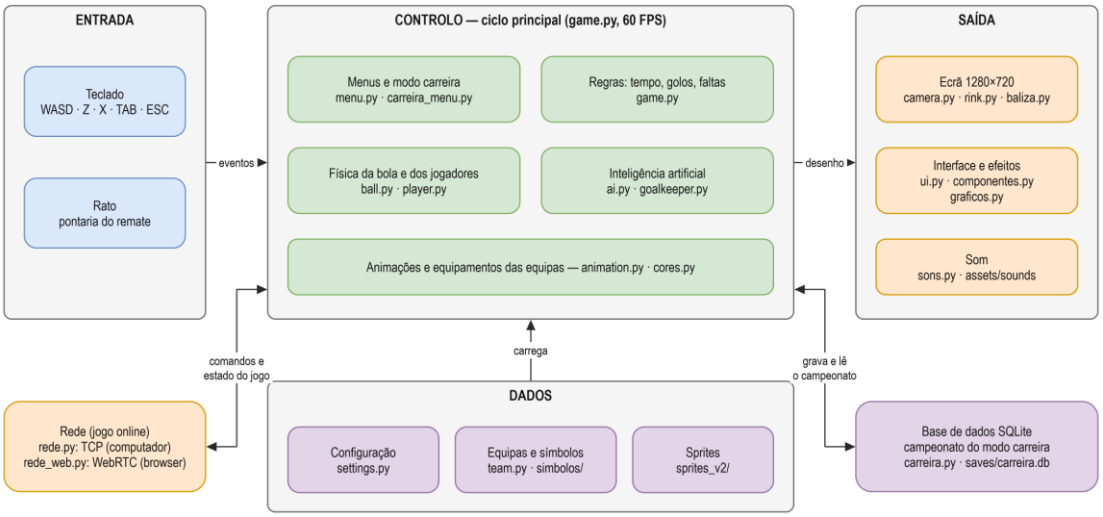


Figura 2 – Arquitetura do sistema em blocos (entrada → controlo → saída)

O diagrama de classes mostra as classes principais e como se ligam. A classe Game é o centro: tem o rinque, a bola, os oito jogadores de campo, os dois guarda-redes, os sete agentes de IA (todos os jogadores menos o humano), a interface e os menus. A Camera pertence ao Rink e é usada para desenhar tudo em perspetiva.

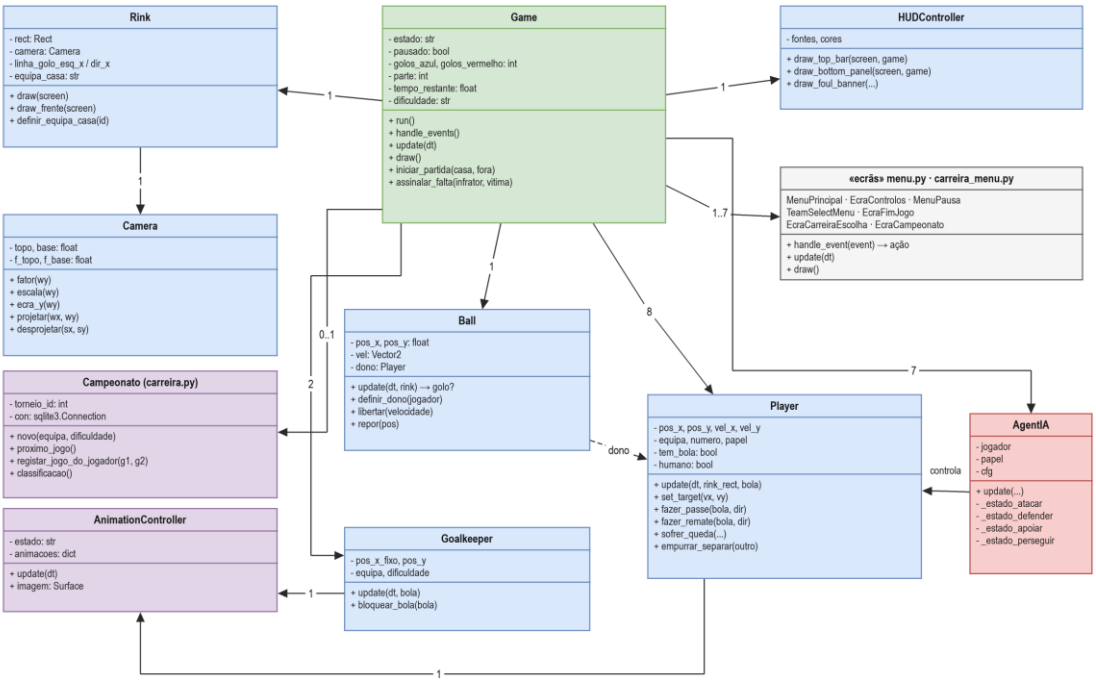


Figura 3 – Diagrama de classes (simplificado)

O modo carreira guarda o campeonato numa base de dados SQLite (ficheiro saves/carreira.db). O modelo de dados mostra as tabelas e as relações: cada torneio pertence à equipa escolhida pelo jogador e tem 28 partidas e uma linha de classificação por equipa; cada partida liga duas equipas (casa e fora). Cada equipa tem 5 jogadores (tabela Jogador) e a tabela Estatistica guarda quantos golos cada jogador marcou em cada partida, o que permite fazer a tabela dos melhores marcadores com uma consulta SQL.

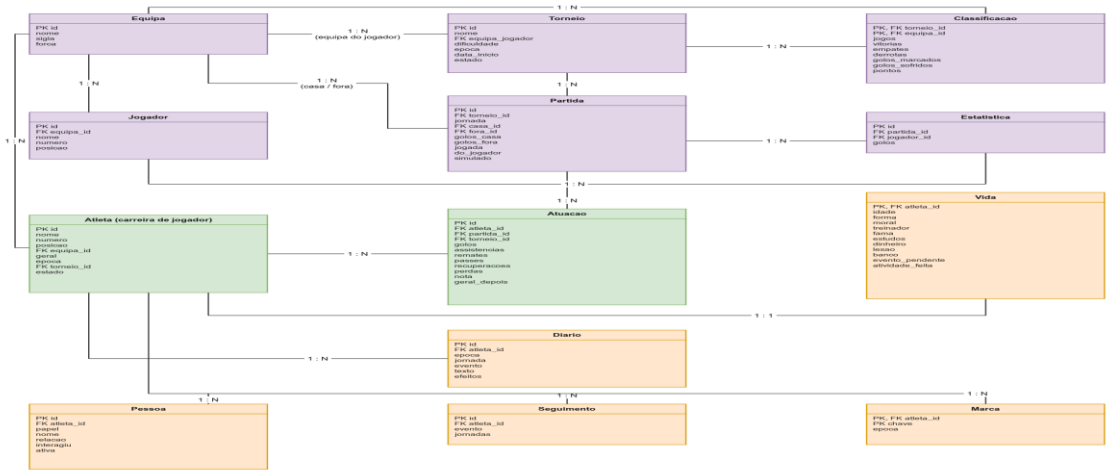


Figura 4 – Modelo de dados da base de dados SQLite (modo carreira e carreira de jogador)

2.4.3 Modelação comportamental (fluxogramas / máquina de estados) ◆ MADS

O jogo funciona como uma máquina de estados: em cada momento está num estado (menu, seleção, partida, pausa, etc.) e as ações do jogador fazem-no mudar de estado. No código isto é a variável estado da classe Game, junto com as variáveis pausado e fim_jogo.

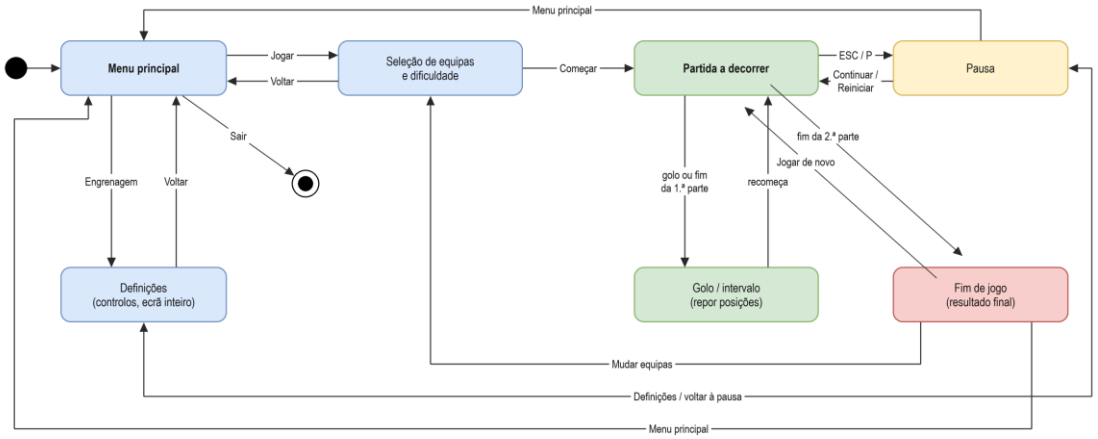


Figura 5 – Máquina de estados do jogo

O fluxograma seguinte mostra o ciclo principal, que se repete 60 vezes por segundo: ler o teclado e o rato, atualizar tudo e desenhar. A física e a lógica usam o tempo que passou desde o frame anterior (dt), para o jogo andar à mesma velocidade em computadores mais rápidos ou mais lentos.

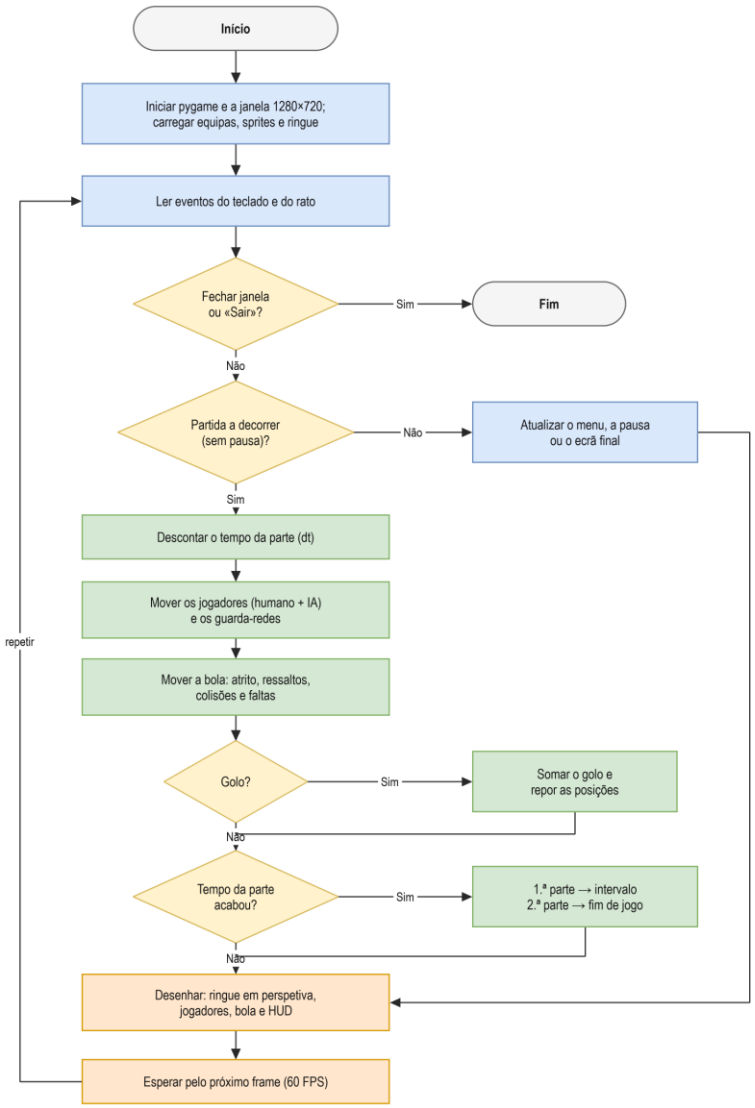


Figura 6 – Fluxograma do ciclo principal do jogo

Cada jogador da IA decide o que fazer em cada frame. Se tem a bola, ataca: contorna a baliza se estiver atrás dela, remata para o canto mais longe do guarda-redes se estiver perto e com a linha livre, passa a um colega desmarcado se estiver pressionado, ou conduz a bola fintando quem aparece à frente. Se não tem a bola e o adversário tem, os jogadores organizam-se numa fila: o mais perto do portador pressiona, o segundo faz a cobertura entre a bola e a baliza e os outros fecham os espaços. Quando é um colega a ter a bola, abrem espaço para receber. Com a bola solta, só vai buscá-la o jogador a quem o passe se destina ou o mais perto, e vai ao ponto onde a bola vai estar, não onde está.

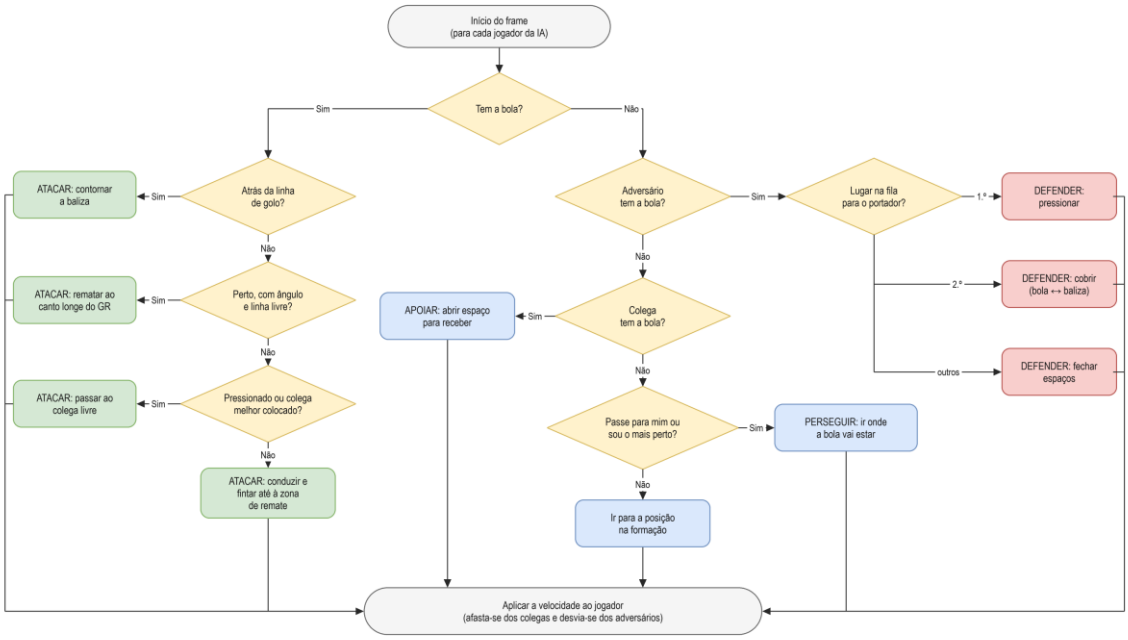


Figura 7 – Fluxograma de decisão da inteligência artificial

O modo carreira acrescenta dois estados ao jogo: a escolha da equipa (carreira_escolha) e o ecrã do campeonato (carreira). O fluxograma seguinte mostra como se passa de um para o outro e para as partidas.

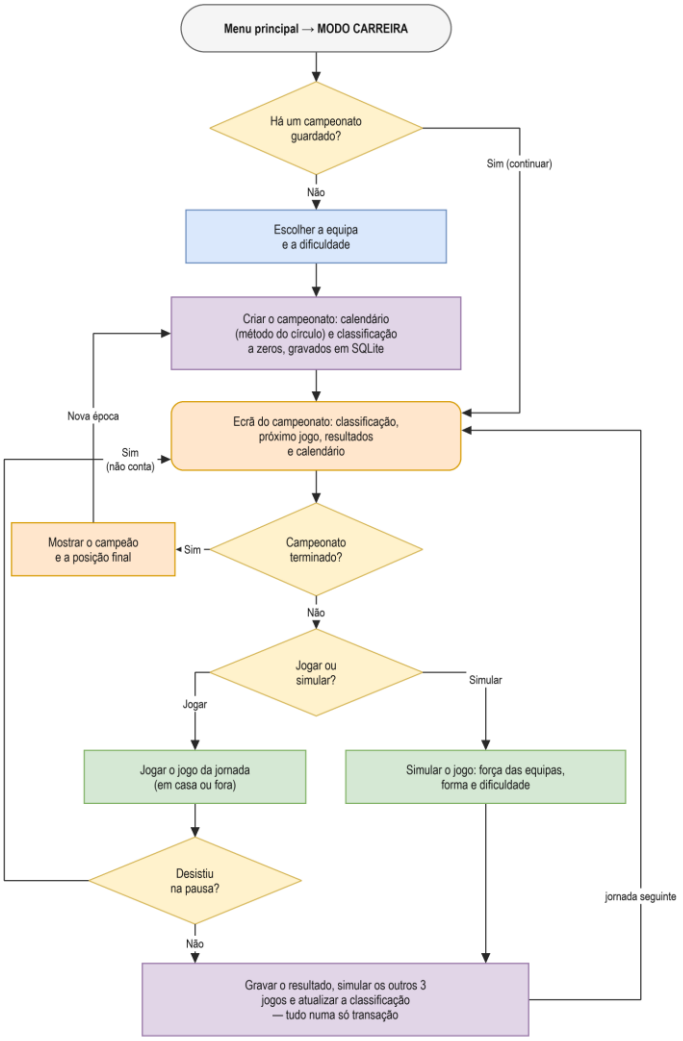


Figura 8 – Fluxograma do modo carreira

2.5 Projeto do hardware

O StickMania é um projeto só de software, por isso não há hardware a construir. Esta secção descreve o equipamento necessário para correr o jogo.

2.5.1 Componentes: sensores, atuadores e controlador

Não há sensores nem atuadores físicos. Os dispositivos usados são os de qualquer computador:

Tabela 2 – Requisitos de hardware e software para correr o jogo

Componente	Mínimo	Recomendado
Processador	Dual-core 2 GHz	Quad-core 2,5 GHz ou superior
Memória RAM	4 GB	8 GB

Componente	Mínimo	Recomendado
Ecrã	1280×720	1920×1080
Periféricos	Teclado	Teclado e rato
Sistema operativo	Windows 10	Windows 10 ou 11
Software	Python 3.10 + pygame-ce	Python 3.12 ou mais recente + pygame-ce

2.5.2 Esquema elétrico e ligações

Não se aplica: o projeto não tem circuitos elétricos.

2.5.3 Estrutura mecânica

Não se aplica: o projeto não tem estrutura mecânica.

2.6 Prototipagem (protótipo físico e/ou simulação) ◆ MADS

Em vez de um protótipo físico, foram feitos protótipos jogáveis do próprio jogo. O primeiro mostrava o rinque visto de cima, com jogadores e bola simples. As versões seguintes acrescentaram a física, a IA, a interface e, por fim, a câmara em perspetiva com bancadas, tabelas com altura e balizas em 3D.

Antes de programar a interface foi feito um mockup (desenho de como o ecrã devia ficar): marcador no topo, mini-mapa, cartão do jogador ativo e controlos em baixo. A figura seguinte mostra o estado atual do jogo.



Figura 9 – Captura de ecrã do StickMania durante uma partida

3. Implementação

3.1 Montagem do hardware

Não há hardware para montar. Em vez disso, foi preparado o ambiente de desenvolvimento:

- **Linguagem:** Python 3 (versão 3.10 ou mais recente).
- **Biblioteca gráfica:** pygame-ce 2.5, para a janela, o desenho, o teclado, o rato e o tempo.
- **Editor:** Visual Studio Code.
- **Controlo de versões:** Git e GitHub (repositório RafaelSilveira23/Stickmaniapap).
- **Diagramas:** draw.io (diagrams.net).
- **Arranque:** o ficheiro jogar.vbs procura um Python compatível com o pygame-ce e abre o jogo sem a janela da consola; se faltar alguma coisa, abre o jogar_com_consola.bat, que instala o pygame-ce e mostra as mensagens.

3.2 Programação do controlo (lógica e algoritmos)

O código está dividido em módulos, um por responsabilidade (ver a arquitetura na secção 2.4.2). Seguem-se os algoritmos mais importantes.

3.2.1 Trabalho feito por Rafael da Hora Silveira

Esta secção descreve em pormenor as partes do jogo programadas por Rafael da Hora Silveira: a física da bola e dos jogadores, os gráficos com a câmara em perspetiva e o jogo online, no computador e no browser.

Física da bola. Em cada frame a bola anda vel × dt. O atrito do piso de madeira multiplica a velocidade por 0,991 quando a bola vai rápida e por 0,975 quando vai lenta (abaixo de 120 px/s), e abaixo de 8 px/s a bola para. Ao bater numa tabela, a componente da velocidade perpendicular à tabela troca de sinal e é multiplicada pelo coeficiente de ressalto 0,78. Um pequeno efeito lateral (spin) torna os ressaltos menos previsíveis. O passe tem força 400 px/s e o remate 650 px/s. Os cantos do rinque são arredondados (raio 50 px): se a bola passa a curva da tabela, volta a ser posta dentro dela e ressalta segundo a normal da curva.

Bola no stick. Quando um jogador tem a bola, ela fica a 22 px do centro, do lado para onde ele olha. Em vez de mudar de lado instantaneamente, o ângulo da bola à volta do jogador roda no máximo 0,24 rad por frame (cerca de 14 rad/s), como se o stick a

arrastasse, e quando alguém apanha a bola ela desliza até ao stick a 9 px por frame. A bola no stick também nunca pode ficar dentro da rede da baliza.

Movimento e colisões dos jogadores. O jogador não muda de velocidade de repente: acelera até à velocidade pretendida (200 px/s, ou 260 px/s com Shift) com uma aceleração de 2000 px/s². As colisões entre jogadores são detetadas pela distância entre os centros (menos de 38 px). Quando chocam, acontecem três coisas: são separados (cada um anda metade da sobreposição); a velocidade com que vinham um contra o outro é anulada, como num choque inelástico, para não ficarem a empurrar-se e a tremer; e voltam a ser mantidos dentro do rinque e fora das balizas.

Desarmes. Quando um defensor choca com quem tem a bola, há uma tentativa de desarme com probabilidade p = 0,42 + (defesa do defensor − 0,5 × velocidade do portador − 30) / 100, limitada entre 15 % e 85 % (o jogador humano tem +18 %). Se ganha, a bola salta para o lado dele. Quem acabou de ganhar a bola fica 0,6 s protegido e quem a perdeu fica 0,8 s sem poder desarmar, o que acabou com o «pingue-pongue» da bola a mudar de dono várias vezes por segundo. Só há falta em choques fortes ou cargas por trás.

Ordem de desenho. A bola entra na mesma ordenação por profundidade que os jogadores: se passar atrás do guarda-redes ou de um jogador, fica tapada por ele. Antes era desenhada sempre por cima de todos, o que fazia parecer que atravessava o corpo do guarda-redes.

Câmara em perspetiva. A física continua num campo plano visto de cima. A câmara só muda o desenho: o campo passa a ser um trapézio, mais estreito ao fundo. Para uma posição t (0 = fundo, 1 = frente), a escala horizontal é f(t) = f_topo + (f_base − f_topo)·t e a posição vertical no ecrã é proporcional ao integral G(t) = f_topo·t + (f_base − f_topo)·t²/2. Para saber onde o rato aponta no campo é preciso fazer a conta ao contrário, o que dá uma equação do 2.º grau, resolvida com a fórmula resolvente. Os jogadores do fundo são desenhados mais pequenos e antes dos da frente (ordenados pela posição y), para ficarem por trás.

Equipamentos das equipas. Os sprites dos jogadores são desenhados com um equipamento azul. Ao carregar a partida, os píxeis com tom (hue) entre 180° e 260° são pintados com as cores do clube, mantendo as sombras. As imagens recoloridas ficam guardadas em cache para não repetir o trabalho.

Animações. Cada jogador tem um controlador de animações com estados (parado, correr, com bola, passe, remate, queda). A velocidade da animação acompanha a velocidade do jogador, e há uma inclinação para a frente ao patinar e um pequeno efeito ao mudar de direção.

Gráficos. Para melhorar a qualidade gráfica sem tornar o jogo mais lento, quase tudo o que é novo é desenhado uma só vez e guardado em cache. O piso passou a ser desenhado com o dobro da resolução e só depois é reduzido (super-amostragem), o que deixa as linhas e o emblema do centro suaves, sem «escadinhas». Cada tábua de madeira tem um tom diferente, veios e alguns nós; junto às tabelas o piso fica mais escuro; e as luzes do pavilhão refletem-se no verniz (manchas de luz somadas às cores). Os jogadores,

os guarda-redes e a bola têm sombras com a borda esbatida. As animações dos jogadores não foram alteradas. Com vsync, a imagem fica sincronizada com o monitor.

Jogo online. Na versão 20 passou a ser possível jogar contra um amigo noutro computador (Figura 10). No menu principal há o botão JOGAR ONLINE: um jogador escolhe CRIAR JOGO e fica à espera (é o anfitrião e joga com a equipa da casa); o outro escolhe ENTRAR NUM JOGO e escreve o endereço IP que aparece no ecrã do anfitrião (joga com a equipa de fora). Depois o anfitrião escolhe as equipas e a partida começa nos dois computadores ao mesmo tempo. O jogo lembra-se do último endereço escrito e aceita CTRL+V para colar o endereço que o amigo enviou.

Anfitrião com autoridade. Só o computador do anfitrião corre a simulação (física, IA, golos e faltas), como num jogo normal; um dos jogadores da equipa de fora deixa de ser da IA e passa a obedecer aos comandos que chegam pela rede. O convidado não simula nada: envia as teclas que carrega (direção, sprint, passe, remate e trocar de jogador) e recebe 30 vezes por segundo o estado do jogo – a posição e a velocidade de cada jogador, a bola, os guarda-redes, o resultado, o tempo, as faltas e os sons e efeitos que aconteceram (Figura 11). Assim os dois veem sempre o mesmo jogo, porque só existe uma simulação. Para não se ver aos saltos entre dois estados, o convidado continua o movimento de cada jogador com a velocidade recebida e corrige a posição aos poucos (suavização); só quando a diferença é muito grande (por exemplo numa reposição depois de um golo) o jogador salta logo para o sítio certo.

Comunicação. A ligação usa TCP na porta 50555 (módulo rede.py). Cada mensagem é um dicionário do Python convertido em JSON, com 4 bytes à frente a indicar o tamanho, porque o TCP envia um fluxo contínuo de bytes e não separa as mensagens. A leitura e o envio correm em threads próprias, ligadas ao jogo por filas, para o jogo nunca ficar parado à espera da rede; se a ligação estiver lenta, os estados antigos são descartados. Um estado ocupa cerca de 1,2 KB, ou seja, cerca de 36 KB/s. Se um dos jogadores sair, o outro recebe uma mensagem de despedida e volta ao ecrã online com o aviso. Na rede local basta o IP; pela internet os dois usam uma VPN para jogos (por exemplo o Radmin VPN), que põe os dois computadores na mesma rede virtual, ou o anfitrião abre a porta no router.

Diferenças no jogo online. O remate na primeira pessoa pára o jogo enquanto se aponta, o que não pode acontecer quando há outra pessoa a jogar. Por isso, online, o X faz um remate direto à baliza, apontado para cima ou para baixo com as setas. Pela mesma razão a pausa não pára o jogo: só abre o menu por cima, com a opção de abandonar. Os colegas e os guarda-redes das duas equipas jogam ao mesmo nível, para ser justo, e no fim do jogo só o anfitrião pode recomeçar ou mudar as equipas.

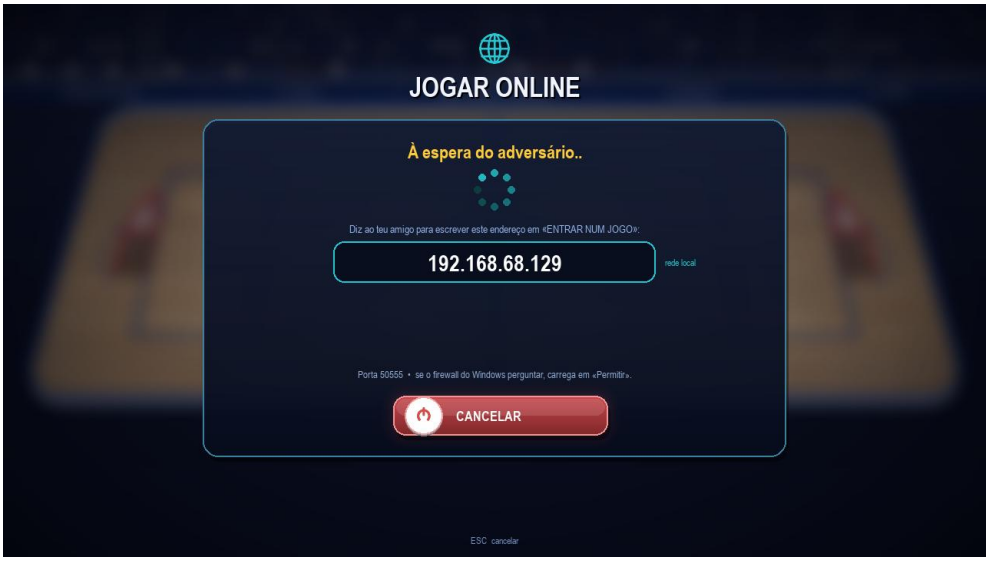


Figura 10 – Jogo online: o anfitrião criou o jogo e espera pelo adversário, com o endereço que o amigo tem de escrever



Figura 11 – Jogo online no computador do convidado: controla um jogador do SL Benfica (cartão em baixo) e vê o jogo que o anfitrião envia

Versão para o browser. Para qualquer pessoa poder jogar sem instalar o Python, criou-se uma versão que corre no browser, publicada no GitHub Pages em rafaelsilveira23.github.io/Stickmaniapap (Figura 13). Usou-se o pygbag, que compila o Python e o pygame-ce para WebAssembly; o código do jogo é praticamente o mesmo. O script ferramentas/construir_web.py junta o jogo e os sprites numa pasta, acrescenta um main.py próprio e chama o pygbag. Foram precisas poucas mudanças: o ciclo principal passou a ter uma versão assíncrona (run_async, que em cada frame devolve o controlo ao browser com await asyncio.sleep(0)), o módulo wave só é importado quando é preciso criar os sons (no browser já vêm feitos), o pygame.font é importado à parte e «Sair» volta a abrir a página. No browser não existe o Arial, por isso usa-se a letra do pygame.

Online no browser. Um browser não pode abrir ligações TCP, por isso o rede.py não funciona lá. Os browsers têm o WebRTC, que liga dois browsers diretamente. Para se

encontrarem é preciso um servidor que só troca os dados iniciais da ligação; usa-se o servidor público e gratuito do PeerJS. Quem cria o jogo recebe um código de 4 letras (Figura 12) e o amigo escreve esse código; depois as mensagens vão diretamente de um browser para o outro. O módulo rede_web.py tem as mesmas classes e funções do rede.py (enviar, mensagens, ativa, terminar), por isso o online.py funciona igual no computador e no browser. A ligação ao JavaScript da página faz-se com platform.window.eval, e o jogo lê as mensagens recebidas em cada frame, sem ficar à espera da rede.

Problemas encontrados no browser. Em ecrãs com zoom (portáteis a 125–200 %) os cliques acertavam no botão de baixo: o pygbag avisa que não suporta esses ecrãs. O módulo navegador.py passou a ler a posição do rato diretamente do browser e a convertê-la para as coordenadas do jogo (x × 1280 / largura do jogo no ecrã), substituindo-a nos eventos. Também se descobriu que no browser o gerador de números aleatórios do Python começava sempre igual, e todos os jogos online recebiam o mesmo código (6TQG); passou a ser baralhado com o relógio e com o Math.random do browser. Por fim, a página ajusta o jogo à janela sem o deformar (16:9, centrado).

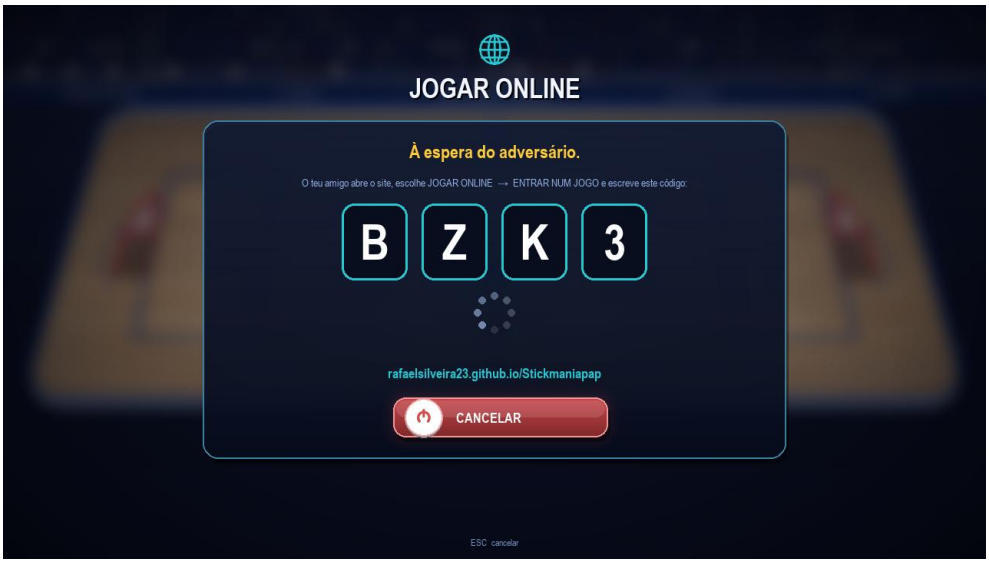


Figura 12 – Versão do browser: quem cria o jogo recebe um código de 4 letras para o amigo escrever



Figura 13 – Jogo online no browser, em rafaelsilveira23.github.io/Stickmaniapap (o convidado controla um jogador do Riba d'Ave)

3.2.2 Trabalho feito em conjunto

Estas partes foram feitas pelos dois elementos do grupo: o ciclo principal do jogo, o modo carreira, o estilo arcade, a otimização e a organização do projeto.

Ciclo principal. game.py tem a classe Game, com o método run() que repete: handle_events() (ler o teclado e o rato), update(dt) (atualizar tudo) e draw() (desenhar). O pygame limita o ciclo a 60 FPS e dt é o tempo, em segundos, desde o frame anterior.

Modo carreira. O jogador escolhe a equipa e a dificuldade dos adversários (Figura 14) e joga um campeonato contra as outras 7 equipas, uma vez contra cada (7 jornadas de 4 jogos). Em cada jornada joga o seu jogo ou carrega em «Simular» (também pode simular a época toda); os outros 3 jogos são sempre simulados. Vitória vale 3 pontos, empate 1 e derrota 0. O código está em carreira.py (dados e regras) e carreira_menu.py (ecrãs).

Calendário. Usa-se o método do círculo: as 8 equipas são postas numa roda, a primeira fica fixa e as outras rodam uma posição em cada jornada; em cada jornada joga a 1.ª contra a 8.ª, a 2.ª contra a 7.ª, etc. Assim cada equipa joga exatamente uma vez contra cada uma das outras (28 jogos). Para decidir quem joga em casa, fica em casa quem tem menos jogos em casa até ali (e, em caso de empate, quem jogou fora na jornada anterior); assim cada equipa tem 3 ou 4 jogos em casa. A ordem das equipas é baralhada em cada época, para o calendário mudar.

Jogos simulados. Cada equipa tem uma força entre 0 e 100 (do FC Porto, 90, ao CRP Lavra, 67). Na primeira versão os golos seguiam uma distribuição de Poisson, mas apareciam resultados sem sentido, como 14-3, e a equipa muito mais fraca ganhava em

11,5 % dos jogos. O modelo novo faz três coisas: (1) soma à força de cada equipa a «forma do dia», um valor aleatório com distribuição normal (desvio padrão 4,5), e a forma recente (+1,5 por cada vitória e −1,5 por cada derrota nos últimos 3 jogos); (2) calcula os golos esperados com golos = 3,1 × e^(0,24 × diferença de forças / 10), mais 0,3 para quem joga em casa; (3) sorteia os golos com uma distribuição binomial (várias oportunidades, cada uma com 50 % de dar golo), que tem a mesma média mas varia menos, e com um máximo de 11. Quando é o jogador a simular o seu jogo, a dificuldade também conta (+5 de força no Fácil, −5 no Difícil). A Tabela 7 mostra o efeito.

Probabilidades e forma. O cartão do próximo jogo mostra as probabilidades de vitória, empate e derrota se o jogo for simulado. São calculadas pelo método de Monte Carlo: o jogo é simulado 600 vezes com o mesmo modelo e conta-se quantas vezes cada resultado acontece. A tabela mostra também a forma das equipas (os últimos 5 resultados como bolinhas verdes, amarelas e vermelhas) e os jogos simulados do jogador têm a etiqueta «SIM» (coluna simulado da tabela Partida).

Planteis. Cada clube tem o seu plantel real, definido em team.py: um guarda-redes e quatro jogadores de campo, como no hóquei em patins (antes o jogo tinha cinco jogadores de campo com nomes inventados e iguais para todos os clubes). Os quatro jogam em losango: um avançado, dois médios nas alas e um defesa. Os nomes aparecem no cartão do jogador ativo, nas faltas e nos golos. Ao arrancar, o modo carreira copia os planteis para a tabela Jogador com um «upsert» (INSERT ... ON CONFLICT DO UPDATE): se um nome mudar em team.py, a base de dados atualiza-se sozinha sem perder os golos das épocas anteriores.

Marcadores. Num jogo jogado, a bola guarda o último jogador de cada equipa que lhe tocou (ao ficar com ela ou num desvio). Quando há golo, o marcador é o último jogador da equipa que marcou a tocar na bola, e o golo fica registado com o minuto. No fim do jogo aparecem os marcadores de cada equipa e, no modo carreira, são gravados na tabela Estatistica. Nos jogos simulados, cada golo é sorteado entre os jogadores de campo, com mais hipóteses para o avançado (40 %) do que para os médios (23 % cada) e o defesa (14 %). A tabela dos melhores marcadores é uma consulta com SUM, GROUP BY e ORDER BY.

Força das equipas no jogo. A força dos clubes também conta nos jogos jogados: a velocidade dos jogadores da IA é multiplicada por 1 + (força − 79) / 23 × 0,10 (cerca de +5 % no FC Porto e −5 % no CRP Lavra) e a pontaria melhora na mesma proporção. A dificuldade continua a ser o que mais conta.



Figura 14 – Modo carreira: escolha da equipa e da dificuldade



Figura 15 – Modo carreira: classificação com a forma, próximo jogo com as probabilidades e resultados da jornada

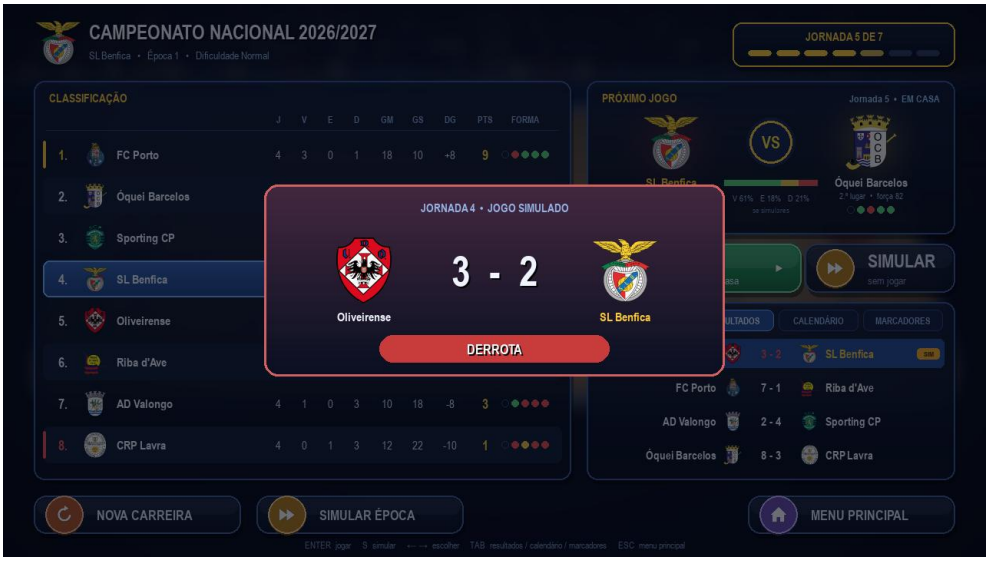


Figura 16 – Modo carreira: resultado de um jogo simulado



Figura 17 – Modo carreira: fim da época, com o campeão e os melhores marcadores

Estilo arcade. Inspirado no estilo dos jogos desportivos arcade (como o Tape to Tape), mas sem copiar imagens, personagens ou regras, foi criado o módulo efeitos.py, que dá mais energia ao jogo sem mexer na física nem na IA. Nos impactos (remate forte, defesa, roubo de bola, bola no poste) o jogo congela durante 40 a 70 ms («hit-stop»), o ecrã treme, saltam faíscas e aparece um texto de banda desenhada («DEFESA!», «POSTE!», «ROUBO!»), que entra a crescer e sobe a desaparecer. A bola deixa um rasto quando vai depressa e as tabelas levantam pó. Os jogadores têm um contorno escuro (a forma do sprite, obtida com uma máscara de píxeis, é desenhada em escuro deslocada em 8 direções). No golo aparece um «GOLO!» gigante numa explosão em estrela na cor da equipa, com o nome de quem marcou, um clarão, confetes e câmara lenta durante 2 segundos, e só depois a bola volta ao centro.

Tempo de jogo e ecrã a tremer. O relógio do jogo passa pelo gestor de efeitos: em cada frame recebe o tempo real e devolve o tempo que o jogo deve andar (0 durante uma pausa,

25 % durante a câmara lenta, voltando aos poucos ao normal). O tremer só desloca a imagem do campo; o marcador e o painel de baixo ficam parados para se lerem bem. O estilo arcade liga-se e desliga-se nas Definições e a escolha fica gravada; com ele desligado o jogo fica exatamente como antes.



Figura 18 – Estilo arcade: festa de um golo, com o nome de quem marcou

Otimização. Com o perfilador do Python (cProfile) mediu-se onde o jogo gasta tempo em cada frame. O jogo já corria bem, mas havia trabalho repetido sem necessidade: o marcador do topo era redesenhado de raiz em todos os frames (9 textos e 2 fontes novas, cerca de 1,5 ms), várias partes criavam a mesma fonte em todos os frames (1 040 fontes em 600 frames, cada uma demora 0,25 ms), o título «GOLO!» do remate era criado e rodado de novo em cada frame, os botões do menu redesenhavam o gradiente do fundo linha a linha e os feixes de luz do menu eram rodados com suavização. Agora o marcador é desenhado numa camada guardada que só é refeita quando algo muda (o tempo ao segundo, um golo, uma falta), as fontes passam pelo módulo fontes.py, que cria cada fonte uma só vez, e os títulos, os fundos dos botões e os feixes ficam guardados ou usam uma rotação mais rápida. Resultado (mediana do desenho de um frame): jogo de 3,85 para 2,35 ms, menu principal de 7,0 para 3,7 ms e carreira de jogador de 4,0 para 2,9 ms. Nas medições descobriu-se ainda uma pequena fuga de memória: a cache de imagens guardava um ícone novo em cada frame da animação dos botões (a cor mudava sempre um pouco); a cor passou a ser arredondada e a cache tem um limite.

Arranque sem consola. O jogo arrancava com o jogar.bat, que deixava aberta a janela preta da consola. Foi criado o jogar.vbs, um pequeno script do Windows que procura um Python 3.10 ou mais recente com o pygame-ce e abre o jogo com o pythonw, a versão do Python que não mostra a consola. O jogo é aberto com a janela normal: na primeira versão pedia-se ao Windows para esconder a janela (como se faz para a consola), mas isso escondia também a janela do próprio jogo, que ficava a correr invisível. Se não encontrar, abre o jogar_com_consola.bat (o antigo jogar.bat), que mostra o que falta e instala o pygame-ce. Como sem consola um erro não se veria, o main.py apanha qualquer erro, grava-o em saves/erro.txt e mostra-o numa janela do Windows.

Limpeza do repositório. Verificou-se no código que ficheiros o jogo carrega de facto e retiraram-se os que nunca eram usados: a pasta sprites_v3 (sprites gerados pelo script ferramentas/gerar_sprites.py, que os volta a criar se for preciso), a versão pixel, as spritesheets, o árbitro e as imagens antigas da interface da sprites_v2, uma captura solta e quatro pastas vazias. O repositório ficou cerca de 4,5 MB mais pequeno; os ficheiros continuam no histórico do Git.

3.2.3 Trabalho feito por Vasco Andreia (resumo)

As partes seguintes foram feitas por Vasco Andreia. Estão aqui só resumidas, para se perceber o jogo completo; estão descritas em pormenor no relatório de Vasco Andreia.

Inteligência artificial e guarda-redes. Cada jogador da IA tem um papel (avançado, médio ou defesa) e em cada frame escolhe entre atacar, defender, apoiar ou perseguir; antes de passar ou rematar verifica se a linha está livre. Os três níveis de dificuldade mudam a velocidade, o tempo de reação e a pontaria. O guarda-redes move-se sobre a linha de golo e reage ao que viu há alguns décimos de segundo, por isso não é perfeito. O código está em ai.py e goalkeeper.py.

Base de dados. O modo carreira e a carreira de jogador guardam tudo numa base de dados SQLite (classificação, jogos, marcadores e o jogador criado); a classificação é lida ordenada por pontos, diferença de golos e golos marcados.

Menus, interface e sons. O menu principal tem botões grandes com ícone e as definições (teclas, ecrã inteiro, estilo arcade e volume) ficam gravadas num ficheiro JSON. Todos os ecrãs usam as mesmas peças (painel de vidro e botões com ícone, em componentes.py), a pausa e o fim de jogo mostram o jogo desfocado por trás e há transições suaves entre ecrãs. Os 14 sons (apito, tacadas, tabela, poste, público e buzina do golo) são criados por síntese em sons.py, a partir de ondas sinusoidais e ruído, sem ficheiros de fora.

Remate na primeira pessoa. Ao carregar em X com a bola, a câmara passa para os olhos do jogador: aponta-se a qualquer ponto da baliza e escolhe-se a força, e os defesas que estão à frente podem bloquear o remate (remate_pov.py). O modo teste deixa treinar o remate só contra o guarda-redes.

Carreira de jogador. O utilizador cria o seu jogador e joga a época só com ele, com uma nota em cada jogo e propostas de outros clubes no fim. Fora do rinque tem uma vida (vida.py): forma, moral, confiança do treinador, fama, estudos e dinheiro, eventos com escolhas e consequências, pessoas com quem se relaciona e conquistas.

3.3 Integração hardware – software

Neste projeto, a integração é entre o jogo e os periféricos do computador. A entrada é lida com os eventos do pygame (teclas premidas e cliques) e com o estado do teclado em cada frame (para o movimento contínuo). A saída é desenhada numa superfície de 1280×720 que é mostrada no ecrã 60 vezes por segundo.

Tabela 3 – Controlos do jogo

Tecla / botão	Ação
W A S D	Mover o jogador ativo
Shift	Sprint (patinar mais depressa)
Z	Passar a bola
X	Remate na primeira pessoa: o rato ou as setas apontam à baliza; manter X (ou o botão do rato) enche a barra de força e largar remata
TAB / C / Espaço	Trocar para o jogador mais perto da bola
TAB / C / Espaço / Z (sem bola)	Na carreira de jogador: pedir a bola ao colega que a tem
V / P / L / C	Na carreira de jogador: atividades da semana / pessoas / diário / conquistas
ESC / P	Pausa (no ecrã do remate, ESC cancela o remate)
R	No MODO TESTE: novo remate noutro sítio
T / Y	Trocar a equipa da casa / de fora
F4	Mudar a dificuldade durante o jogo
F11	Ecrã inteiro (também nas Definições, na engrenagem do menu)
F3	Mostrar informação de depuração

3.4 Decisões técnicas e dificuldades ultrapassadas

Seguem-se as decisões e as dificuldades das partes feitas por Rafael da Hora Silveira e das partes feitas em conjunto. As das partes feitas por Vasco Andreia estão no relatório de Vasco Andreia.

Perspetiva sem mudar a física. A primeira ideia foi transformar toda a física para 3D. Em vez disso, a física ficou num campo plano e só o desenho passou pela câmara. Assim a IA e as colisões não mudaram e não apareceram erros novos.

Balizas desenhadas no código. Os PNG antigos das balizas estavam desenhados de lado e não combinavam com o rinque. As balizas passaram a ser desenhadas no código a partir de pontos 3D projetados pela câmara, em duas camadas: a rede de trás (desenhada antes dos jogadores) e os postes da frente (desenhados depois). Assim a bola e o guarda-redes parecem estar dentro da baliza.

Desempenho. Desenhar o rinque em perspetiva linha a linha é lento. Por isso, o rinque, as tabelas e as bancadas são desenhados uma só vez, quando a partida começa, e depois só se copia essa imagem em cada frame. O jogo arranca em menos de um segundo e mantém os 60 FPS.

Pontaria com o rato. Com a câmara, a posição do rato no ecrã já não é a posição no campo. Foi preciso escrever a função inversa da câmara (desprojetar), com a equação do 2.º grau descrita na secção 3.2.

Sprites. Foi criada uma folha de sprites nova com mais animações, mas o resultado visual ficou pior do que o original e a alteração foi revertida com o Git. A melhoria dos sprites fica para uma fase seguinte.

Cabelo igual em todos os frames. Nos sprites do jogador azul o cabelo aparecia ora castanho, ora preto, conforme o frame, o que se notava ao mudar de animação. Para não refazer os desenhos, fez-se um pequeno programa que percorre a zona da cabeça de cada frame e troca os píxeis castanhos (tom quente e mais escuros do que a pele) por um castanho-escuro quase preto, mantendo o sombreado. Os 26 frames corrigidos ficaram na pasta sprites_v2/hd/frames/novo (os originais em frames/azul não foram apagados) e há também uma folha completa em sprites_v2/hd/spritesheet_nova.png. O jogo passou a carregar os sprites dessa pasta. O alinhamento pelos patins não precisou de ser mudado nos ficheiros, porque o jogo já centra cada frame pelos patins ao carregá-lo. Também se reparou que, quando o jogador estava parado, o stick desaparecia: os desenhos «parado» estão de frente e sem stick. A animação de parado passou a usar dois frames de lado com o stick no chão (os de conduzir a bola, que não têm bola desenhada), trocados devagar para parecer a respiração. Por fim, viu-se que os frames têm 128×128 píxeis mas o boneco ocupa só a zona central (cerca de 74×91 píxeis); como tudo é reduzido para 52×52, os jogadores ficavam pequenos no rinque. Ao carregar os frames, depois de os alinhar pelos patins, o jogo passou a cortar um quadrado de 94×94 píxeis à volta do boneco antes de o reduzir: os jogadores ficaram cerca de 36% maiores no ecrã, sem mudar o tamanho das colisões, e a ponta do stick ficou mais perto da bola.

Bola presa e jogadores colados. Ao início a bola atravessava as tabelas quando ia muito rápida e os jogadores ficavam presos uns nos outros. Foi corrigido limitando a velocidade, verificando a posição depois de cada movimento e separando os jogadores que se sobrepõem.

Colisões e IA com comportamentos estranhos. Mesmo depois disso, os jogadores ainda «tremiam» quando iam uns contra os outros, a bola mudava de dono várias vezes por segundo e a IA tinha comportamentos estranhos: os avançados não voltavam para defender, ninguém ia a uma bola parada entre dois colegas, e quase não havia golos porque o guarda-redes tapava a baliza toda e reagia instantaneamente. Para resolver isto sem «adivinhar», foi criado um script que joga partidas IA contra IA sem janela e conta os problemas (Tabela 5). Cada correção foi medida antes e depois. Também se descobriu que, depois de qualquer golo, a bola ia sempre para a equipa do jogador (agora vai para quem sofreu o golo) e que a dificuldade «Difícil» também melhorava os colegas do jogador.

Bola e guarda-redes. Ao jogar, ainda se viam problemas com a bola e o guarda-redes. Foi feito um segundo script que procura situações estranhas (Tabela 6): a bola saltava até 50 px de um frame para o outro quando o jogador mudava de direção; jogadores ficavam em cima do guarda-redes porque não chocavam com ele; a bola parecia atravessar o guarda-redes por ser sempre desenhada por cima; e, em casos raros, a bola entrava nos cantos arredondados ou ficava no stick dentro da rede. As soluções estão descritas na secção 3.2.

Regras do modo carreira. No modo carreira o jogador controla sempre a sua equipa, quer jogue em casa quer fora; nos jogos fora o rinque passa a ter o emblema e as cores do adversário. Se o jogador desistir a meio (na pausa), o jogo não conta e pode ser jogado

outra vez; não há «reiniciar» nem troca de equipa ou de dificuldade durante o jogo, para o resultado ser justo.

Quatro jogadores de campo. Com quatro jogadores de campo (as regras reais) o jogo ficou com menos golos: entre duas equipas da IA passou de 3,0 para 0,6 golos em 3 minutos, porque há menos ressaltos e o guarda-redes vê todos os remates. Medindo com o script da Tabela 5, foram feitos três ajustes: os médios e o defesa sobem mais no ataque, a IA remata de um pouco mais longe (255 px no nível normal, antes 225) e a antecipação do guarda-redes no nível normal passou de 60 % para 56 %. O resultado foi 1,8 golos em 3 minutos, cerca de 6 por jogo, e menos bolas a trocar de dono várias vezes seguidas (de 18 para 6).

Afinar a simulação com dados. Para escolher os valores do modelo de simulação não se «adivinhou»: foi criado o script ferramentas/analise_simulacao.py, que simula 600 épocas e mede golos por jogo, empates, goleadas, surpresas e quem é campeão. Os parâmetros foram ajustados até os resultados parecerem os de um campeonato real.

Porquê um anfitrião com autoridade. Pensou-se primeiro em pôr os dois computadores a simular o jogo ao mesmo tempo, trocando só as teclas. Mas a física e a IA usam números aleatórios e o tempo de cada frame, e uma diferença mínima fazia os dois jogos separarem-se ao fim de poucos segundos. Com uma só simulação (a do anfitrião) isso não pode acontecer. Escolheu-se TCP em vez de UDP porque garante que as mensagens chegam todas e pela ordem certa, o que simplifica o código; para dois jogadores e 30 estados por segundo o atraso é pequeno.

Porquê o pygbag e o WebRTC. Reescrever o jogo em JavaScript seria refazer tudo. O pygbag deixa usar o mesmo código Python no browser. Para o online pensou-se num servidor próprio (com WebSockets), mas teria de estar sempre ligado num serviço de alojamento; com o WebRTC os browsers ligam-se diretamente e o PeerJS gratuito só serve para se encontrarem. A desvantagem é que algumas redes (por exemplo redes de empresas) bloqueiam ligações diretas; nesse caso o jogo mostra um aviso.

4. Testes e validação

4.1 Plano de testes

Os testes foram de dois tipos: testes manuais (jogar e verificar cada funcionalidade) e testes automáticos, com scripts que arrancam o jogo sem janela (SDL_VIDEODRIVER=dummy), simulam teclas e verificam o resultado. A tabela resume o plano de testes.

Tabela 4 – Plano de testes

N.º	Teste	Resultado esperado	Resultado
T01	Arrancar o jogo com jogar.bat	Abre o menu principal	Passou
T02	Escolher equipas e dificuldade e começar	A partida começa com as equipas escolhidas	Passou
T03	Mover o jogador com WASD e Shift	Move-se nas 8 direções e acelera com Shift	Passou
T04	Passar (Z) e rematar (X)	A bola sai na direção escolhida	Passou
T05	Bola contra as tabelas a alta velocidade	Ressalta e nunca sai do rinque	Passou
T06	Bola a entrar na baliza	Golo somado à equipa certa e bola ao centro	Passou
T07	Fim da 1.ª e da 2.ª parte	Intervalo e depois ecrã de resultado final	Passou
T08	Pausar e continuar	Marcador e tempo mantêm-se	Passou
T09	Trocar equipas durante o jogo (T / Y)	Símbolos, cores e equipamentos mudam	Passou
T10	Rato em perspetiva (projetar e desprojetar)	Erro de 0 px na conversão ida e volta	Passou
T11	Desempenho	60 FPS estáveis durante a partida	Passou
T12	Jogadores a chocar uns com os outros	Separam-se sem tremer nem ficar sobrepostos	Passou
T13	Disputa de bola entre dois jogadores	A bola não muda de dono várias vezes seguidas	Passou
T14	IA defende quando o adversário tem a bola	O mais perto pressiona e outro faz cobertura	Passou
T15	Golo marcado	Recomeça com a bola a equipa que sofreu o golo	Passou
T16	Mudar a dificuldade	Só muda a equipa adversária	Passou
T17	Jogador muda de direção com a bola	A bola roda à volta dele, sem saltos	Passou
T18	Remate rápido junto ao guarda-redes	Não atravessa o GR; a defesa manda a bola para a frente	Passou
T19	Jogador a ir contra o guarda-redes	Não fica em cima do GR	Passou
T20	Bola nos cantos do rinque	Ressalta na curva da tabela	Passou
T21	Calendário do campeonato	7 jornadas, 28 jogos, cada par joga 1 vez, 3 ou 4 jogos em casa	Passou
T22	Época completa no modo carreira	Pontos = 3×V + E; golos marcados = sofridos no total	Passou
T23	Fechar e voltar a abrir o jogo	O campeonato continua no mesmo ponto	Passou
T24	Gravar golos negativos	Recusado e nada é gravado (transação)	Passou
T25	Desistir de um jogo na pausa	O jogo não conta e pode ser jogado outra vez	Passou
T26	Nova época / nova carreira	Época 2 com calendário novo; nova carreira pede confirmação	Passou
T27	Clicar na engrenagem / ↑ e ENTER no menu	Abre as Definições; ESC volta ao menu	Passou
T28	Ecrã inteiro (interruptor e F11)	Muda para ecrã inteiro e volta; a opção fica gravada	Passou
T29	Tempo de desenho de cada frame	Menu 7,6 ms e partida 5,4 ms (menos de 16,7 ms = 60 FPS)	Passou
T30	Sons durante uma partida automática	Apitos, passes, remates, defesas e público a tocar	Passou
T31	Volume dos efeitos e do público	Muda com ← → e fica gravado	Passou

N.º	Teste	Resultado esperado	Resultado
T32	Simular o jogo do jogador (tecla S ou botão)	Resultado gravado com simulado = 1; tabela certa	Passou
T33	Simular a época	Pede confirmação; joga as jornadas que faltam	Passou
T34	Abrir uma gravação da versão anterior	A coluna simulado é acrescentada; nada se perde	Passou
T35	Modelo de simulação (600 épocas)	Resultados realistas (Tabela 7)	Passou
T36	Desenho do ecrã do campeonato	4,7 ms por frame (antes da cache era 25 ms)	Passou
T37	Planteis de todos os clubes	4 jogadores + GR com os nomes certos; mudam ao trocar de equipa	Passou
T38	Golo num jogo jogado	Marcador e minuto certos na mensagem e no fim do jogo	Passou
T39	Marcadores no modo carreira	Soma dos golos dos jogadores = soma dos golos dos resultados	Passou
T40	Jogabilidade com 4 jogadores	1,8 golos e 33 remates em 3 min (IA contra IA)	Passou
T41	Efeitos arcade num jogo IA contra IA	Aparecem defesa, poste, roubo, rasto e golo	Passou
T42	Interruptor «Estilo arcade» nas Definições	Liga e desliga com teclado e rato; fica gravado	Passou
T43	Golo com e sem estilo arcade	Com: festa e câmara lenta; sem: faixa normal e reposição logo	Passou
T44	Desenho com os efeitos	4,7 ms por frame (95 % dos frames abaixo de 11,5 ms)	Passou
T45	X abre o remate na primeira pessoa	Mira com o rato no ponto certo (erro < 1,5 px)	Passou
T46	Carregar e largar X	A barra enche; ao largar remata e mostra o resultado	Passou
T47	ESC no ecrã do remate	Cancela e o jogador continua com a bola	Passou
T48	Tempo limite num jogo (5 s)	O remate sai sozinho	Passou
T49	Remate atrás da linha de golo	Remate rápido sem abrir o ecrã	Passou
T50	MODO TESTE: 31 remates seguidos	7 golos, 18 defesas, 6 fora; volta sempre a ter a bola	Passou
T51	Desenho do ecrã do remate	2 ms a apontar e 3,7 ms no remate (antes 15–20 ms)	Passou
T52	Remate a correr	Velocidade passa a 50 % e o jogador avança (62 px em 0,5 s)	Passou
T53	Defesas no ecrã do remate	Os 3 defesas à frente aparecem; o de trás não	Passou
T54	Apontar através de um defesa	Aviso «LINHA TAPADA» e remate BLOQUEADO	Passou
T55	Depois do remate	O jogador fica onde chegou, com a velocidade a 50 %	Passou
T56	Desenho com a câmara a andar	8,6 ms por frame (antes 12,4 ms)	Passou
T57	Ecrã do remate com o pavilhão	Bancada, marcador com o resultado real e placas em perspetiva	Passou
T58	Sprite do stick	Fundo tirado, letras legíveis, bola pousada na pá e a sair sem saltos	Passou
T59	Desenho com o pavilhão novo	11,5 ms com a câmara a andar e 5,6 ms parado	Passou
T60	Animação da stickada	Stick vem para trás, varre até à bola, a bola só sai no contacto	Passou

N.º	Teste	Resultado esperado	Resultado
T61	Desenho durante o golpe	Média 6 ms e máximo 10,8 ms (antes 38,6 ms)	Passou
T62	Tamanho dos defesas ao avançar	A altura cresce sempre com 1/z, sem limite; não desaparecem enquanto estão à frente	Passou
T63	Defesa mesmo à frente	O jogador trava a 2,8 m; o defesa fica meio transparente, com contorno, anel e nome	Passou
T64	Desenho com 4 defesas	Mediana 5,8 ms e p95 25 ms, igual à versão anterior (6,7 / 25,2 ms)	Passou
T65	Criar jogador	Fica gravado em SQLite e entra no plantel no lugar de um jogador da mesma posição	Passou
T66	Controlo só do meu jogador	75 s de jogo sem nenhuma troca de controlo; TAB / C pedem a bola ao colega	Passou
T67	Época e propostas	Nota média 7,5: propostas do Benfica e do Sporting; na época 2 joga no clube novo	Passou
T68	Outros modos sem alterações	Jogo rápido e modo carreira continuam a trocar de jogador e sem o cartão do jogador	Passou
T69	Eventos e escolhas	O evento abre sozinho, a escolha aplica os efeitos e fica no diário; não se pode jogar sem responder	Passou
T70	Vida no jogo	Com forma 42 o remate em campo é 71 em vez de 72; lesionado não entra em jogo e a jornada é simulada sem ele	Passou
T71	Atividades	Só uma por semana; as que custam dinheiro ficam bloqueadas sem saldo	Passou
T72	Equilíbrio em 6 épocas	No fim de cada época, forma, moral e treinador ficaram entre 40 e 82; 2 lesões em 42 jornadas; propostas em 5 das 6 épocas	Passou
T73	Transições entre ecrãs	Fade de 0,28 s em todas as mudanças de ecrã; o jogo continua a desenhar em 7 a 8 ms por frame	Passou
T74	Abrir a pausa	Primeiro frame da pausa: de 260 ms para cerca de 60 ms (emblemas guardados já limpos)	Passou
T75	Fim de jogo	Vencedor com a fita, quem perdeu mais escuro, marcadores e confetes quando a equipa do jogador ganha	Passou
T76	Nada se partiu	Jogo com pausa e 7 remates, carreira e vida do jogador sem erros	Passou
T77	Todos os eventos	As 42 × 2 ou 3 opções correm sem erros e os textos ficam com os nomes certos	Passou
T78	Pessoas	Uma interação por pessoa por semana (a segunda é bloqueada); a relação sobe e a moral também	Passou
T79	Consequências	Responder à letra nas redes traz o processo disciplinar na jornada seguinte	Passou
T80	Cinco épocas seguidas	10 das 14 conquistas desbloqueadas, mudança para um grande e nenhum erro	Passou

N.º	Teste	Resultado esperado	Resultado
T81	Desenho de um frame	Jogo: mediana 3,85 → 2,35 ms (p95 6,1 → 4,6 ms); menu principal 7,0 → 3,7 ms	Passou
T82	Fontes	Nenhuma fonte criada durante o jogo (antes eram 1 040 em 600 frames)	Passou
T83	Arranque sem consola	O jogar.vbs encontra o pythonw com o pygame-ce e a janela do jogo aparece sem o cmd; um erro aparece numa janela e fica em saves/erro.txt	Passou
T84	Depois da limpeza	Os 9 ecrãs, o jogo, a carreira e a vida funcionam sem os ficheiros retirados	Passou
T85	Criar e entrar num jogo online	O convidado liga-se ao endereço do anfitrião; o anfitrião passa à escolha das equipas e o convidado fica à espera	Passou
T86	Partida online	Começa nos dois computadores com as mesmas equipas; o convidado controla um jogador da equipa de fora e o controlo passa para quem apanha a bola	Passou
T87	Sincronização	Bola e jogadores no mesmo sítio nos dois ecrãs (diferença de poucos píxeis, da suavização), medido durante 30 s	Passou
T88	Golo, sons e efeitos online	O golo, o resultado, a festa e os sons aparecem também ao convidado	Passou
T89	Fim e jogar de novo	O fim do jogo aparece aos dois; o anfitrião recomeça e o convidado entra no jogo novo	Passou
T90	Pausa online	O menu abre, mas o jogo continua (o tempo andou 1,0 s em 1 s de pausa)	Passou
T91	Saída de um jogador	O outro volta ao ecrã online com «O outro jogador saiu do jogo.»	Passou
T92	Endereço errado	Aviso claro (endereço desconhecido, porta inválida ou anfitrião sem jogo) e pode tentar-se outra vez	Passou
T93	Modos sem rede	Jogo rápido, modo teste e carreira de jogador funcionam como antes	Passou
T94	Abrir o site	O jogo carrega no browser (Chrome/Edge) a partir do link, sem instalar nada	Passou
T95	Jogo no browser	Menus, jogo rápido e partida funcionam como no computador	Passou
T96	Cliques em ecrã com zoom	Com o ecrã a 200 % o clique acerta no botão certo (antes acertava no de baixo)	Passou
T97	Código de sala	Cada jogo recebe um código diferente (antes era sempre 6TQG)	Passou
T98	Online no browser	Dois browsers ligam-se com o código; o anfitrião começa e o convidado recebe a partida e controla o seu jogador	Passou
T99	Código errado	Aviso «Não existe nenhum jogo com esse código» e pode tentar-se outra vez	Passou

4.2 Ensaios, calibração e medições

Foram feitas medições com o jogo a correr sem limite de FPS, em simulação automática:

- **Tempo de arranque:** cerca de 0,75 s até a partida estar pronta (inclui desenhar o rinque em perspetiva).
- **Velocidade de desenho:** entre 250 e 290 frames por segundo sem limite, muito acima dos 60 FPS necessários.
- **Precisão da câmara:** converter um ponto do campo para o ecrã e de volta dá erro de 0,0 píxeis.
- **Golos:** em simulações com remates automáticos, todos os golos foram detetados e atribuídos à equipa certa.

Também foram afinados vários valores por tentativa e erro: o coeficiente de ressalto passou de 0,72 para 0,78 (a bola parecia «morta»), a velocidade de sprint passou de 330 para 260 px/s (era demasiado rápida), o tamanho da bola passou de 14 para 11 px e o atrito foi dividido em dois valores (bola rápida e bola lenta) para a bola parar de forma mais natural.

Para a jogabilidade foi criado um script de medição que põe as duas equipas controladas pela IA a jogar sem janela, 8 jogos de 3 minutos, sempre com as mesmas sementes aleatórias (para a comparação ser justa), e conta os problemas. A tabela compara a versão antiga com a versão corrigida.

Tabela 5 – Medição da jogabilidade antes e depois das correções (média por jogo de 3 min, IA contra IA)

Medida	Antes	Depois
Golos	0,4	2,5
Remates	20	51
Bola roubada e devolvida em menos de 0,6 s («pingue-pongue»)	101	18
Passes (muitos eram intercetados)	378	81
Faltas apitadas	15,1	0,6
Frames com jogadores a «tremer» (inverter o sentido)	59	39
Momentos com jogadores sobrepostos	4,8	0,2
Momentos com jogadores fora do rinque	0,4	0

Um segundo script procura problemas da bola e do guarda-redes, em 4 jogos de 3 minutos (IA contra IA):

Tabela 6 – Bola e guarda-redes antes e depois das correções (média por jogo de 3 min)

Medida	Antes	Depois
Saltos da bola de mais de 25 px num frame	157	0
Frames com um jogador em cima do guarda-redes	107	3,5
Frames com a bola fora dos cantos arredondados	0,75	0

Medida	Antes	Depois
Frames com a bola no stick dentro da rede	4	0
Bola a passar junto ao GR sem defesa (a 22–24 px, fora do alcance)	1,0	0,5

Para o modo carreira, o script ferramentas/analise_simulacao.py simulou 600 épocas completas com o modelo antigo (Poisson) e com o novo:

Tabela 7 – Modelo de simulação dos jogos: antigo e novo (600 épocas, 16 800 jogos)

Medida	Antigo	Novo
Golos por jogo	6,6	6,9
Empates	12,9 %	12,8 %
Surpresas (a equipa 12+ pontos mais fraca ganha)	11,5 %	6,3 %
Goleadas (6 ou mais golos de diferença)	8,4 %	7,2 %
Maior número de golos de uma equipa	18	11
FC Porto (força 90) campeão	38,6 %	43,8 %
CRP Lavra (força 67): posição média	7,30	7,33

Para o remate na primeira pessoa simularam-se 1200 remates para cada caso. A primeira versão do modelo era desequilibrada: a meia distância nem os remates aos cantos passavam dos 20 % e com muita força a bola ia para fora em até 64 % das vezes; de perto, os cantos davam 95 % de golos. Depois de afinar a leitura do guarda-redes, a velocidade de mergulho e o erro do remate, os resultados no nível normal (força de 65 %) ficaram assim:

Tabela 8 – Remate na primeira pessoa: percentagem de golos no nível normal (força 65 %)

Distância	Meio	Canto de baixo	Canto de cima	Lado, meia altura
5,3 m	0 %	36 %	90 %	35 %
8,8 m	6 %	29 %	33 %	36 %
13,3 m	7 %	17 %	20 %	23 %

Rematar para o meio quase nunca dá golo; os cantos compensam, sobretudo os de cima de perto; e de longe é mais difícil. No MODO TESTE, 31 remates seguidos deram 7 golos (23 %). Com três defesas entre o jogador e a baliza, um deles mesmo no meio da linha para a baliza, 40 remates aos cantos de cima deram 36 bloqueados e 4 ao poste: com um defesa à frente é preciso mudar de ângulo (avançar ou rematar mais de lado) antes de rematar, como num jogo a sério.

4.3 Resultados obtidos e ajustes

Todos os testes (T01 a T99) passaram. O modo carreira foi testado com um script que joga uma época inteira como se fosse um jogador (carregando nas teclas), com uma base de dados temporária, e verifica a classificação, a gravação e a passagem entre os ecrãs.

Os sons não se podem ouvir num teste automático, por isso o teste T30 «espia» o gestor de som: conta que sons foram pedidos durante 2 minutos de jogo IA contra IA (por exemplo, 22 passes, 12 remates, 4 defesas e os apitos de início, faltas, intervalo e fim). Os testes correm com o controlador de áudio «dummy», que não faz barulho.

O primeiro calendário tinha os jogos em casa mal distribuídos (uma equipa nunca jogava em casa); foi corrigido escolhendo para casa a equipa com menos jogos em casa.

Os testes levaram a vários ajustes: o triângulo de seleção e a etiqueta «GR» ficavam a flutuar acima dos jogadores depois da câmara em perspetiva e passaram a usar o retângulo visível do sprite; o stick ficava demasiado claro e foi escurecido; as tabelas e as balizas foram redesenhadas para ficarem iguais à imagem de referência; e a IA foi afinada para haver menos quedas.

Na afinação do guarda-redes testaram-se várias combinações de alcance, antecipação e tempo de reação. Com o alcance antigo (34 px) o guarda-redes tapava a baliza toda e quase não havia golos; com antecipação perfeita da trajetória também não. Uma primeira versão com um erro de leitura aleatório fazia o guarda-redes mexer-se às vezes para o lado errado num remate direto, o que parecia um erro, e foi trocada pela antecipação parcial. O número de golos muda muito com este valor (com 55 % havia 6 golos em 3 minutos, com 62 % só 1), por isso escolheu-se 60 % no nível normal: cerca de 5 % dos remates dão golo entre duas equipas da IA. Com a passagem a quatro jogadores de campo este valor foi baixado para 56 % (ver a secção 3.4).

5. Gestão do projeto

5.1 Planeamento e cronograma ◆ MADS

A tabela seguinte mostra o cronograma do projeto, desde o anteprojeto até à defesa.

Tabela 9 – Cronograma do projeto

Período	Tarefa	Estado
set.–out. 2026	Anteprojeto, levantamento de requisitos, escolha das tecnologias	Concluído
out. 2026	Protótipo: rinque, jogadores, bola e física básica	Concluído
out. 2026	Interface do mockup, símbolos dos clubes e seleção de equipas	Concluído
out. 2026	Mira de remate, IA com dificuldades, faltas	Concluído
out. 2026	Animações, balizas, câmara em perspetiva e bancadas	Concluído
out. 2026	Correção das colisões, nova IA tática e guarda-redes com reação	Concluído
out. 2026	Correção da bola no stick, das defesas do guarda-redes e da ordem de desenho	Concluído

Período	Tarefa	Estado
nov.–dez. 2026	Estatísticas da partida, modo de 2 jogadores	Planeado
nov. 2026	Modo carreira: campeonato com classificação guardado em SQLite	Concluído
nov. 2026	Menu principal novo, definições, ecrã inteiro e melhoria gráfica	Concluído
nov. 2026	Sons, simulação realista dos jogos e novos ecrãs do modo carreira	Concluído
nov. 2026	Planteis reais (4 + GR), marcadores e melhores marcadores	Concluído
nov. 2026	Estilo arcade (efeitos nos golos e impactos)	Concluído
nov. 2026	Remate na primeira pessoa e MODO TESTE	Concluído
nov. 2026	Fase de remate em movimento, defesas e bloqueio	Concluído
nov. 2026	Ecrã do remate como a imagem de referência (pavilhão, placas, stick)	Concluído
nov.–dez. 2026	Defesas do ecrã do remate: tamanho certo em perspetiva, sprites HD e destaque de quem tapa a linha	Concluído
dez. 2026	Carreira de jogador: criar o jogador, jogar só com ele, nota, evolução e propostas	Concluído
dez. 2026	Vida do jogador: escolhas, eventos, atividades, lesões e banco	Concluído
dez. 2026–jan. 2027	Menus e gráficos: seleção, pausa e fim de jogo no estilo novo, transições entre ecrãs	Concluído
jan. 2027	Vida do jogador: pessoas, consequências, bens e conquistas	Concluído
jan. 2027	Otimização, arranque sem consola e limpeza do repositório	Concluído
fev. 2027	Modo online: jogar contra um amigo noutro computador (rede local ou internet)	Concluído
fev. 2027	Versão para o browser no GitHub Pages e jogo online com código (WebRTC)	Concluído
jan.–mar. 2027	Mais estatísticas (remates, assistências) e transferências	Planeado
mar.–abr. 2027	Testes com colegas, correção de erros, novos sprites	Planeado
abr.–mai. 2027	Relatório final, manual de utilizador e apresentação	Em curso
jun. 2027	Defesa da PAP	Planeado

A Figura 19 mostra o mesmo cronograma num diagrama de Gantt, com a divisão de tarefas entre os dois elementos do grupo. Cada barra é uma tarefa, colocada no mês em que foi feita ou em que está prevista. A coluna «Quem fez» e a cor da barra mostram quem fez a tarefa: azul para o Vasco Andreia, laranja para o Rafael da Hora Silveira e verde para as tarefas feitas pelos dois. O padrão da barra mostra o estado: cheia para concluído, com riscas para em curso e só com o contorno para planeado. As tarefas que juntavam partes dos dois (por exemplo, a correção das colisões e a IA tática) aparecem em duas linhas, uma para cada um. Vê-se que o desenvolvimento foi feito em versões curtas e seguidas (uma por semana, mais ou menos) e que os últimos meses ficam reservados para testes, relatório e defesa.

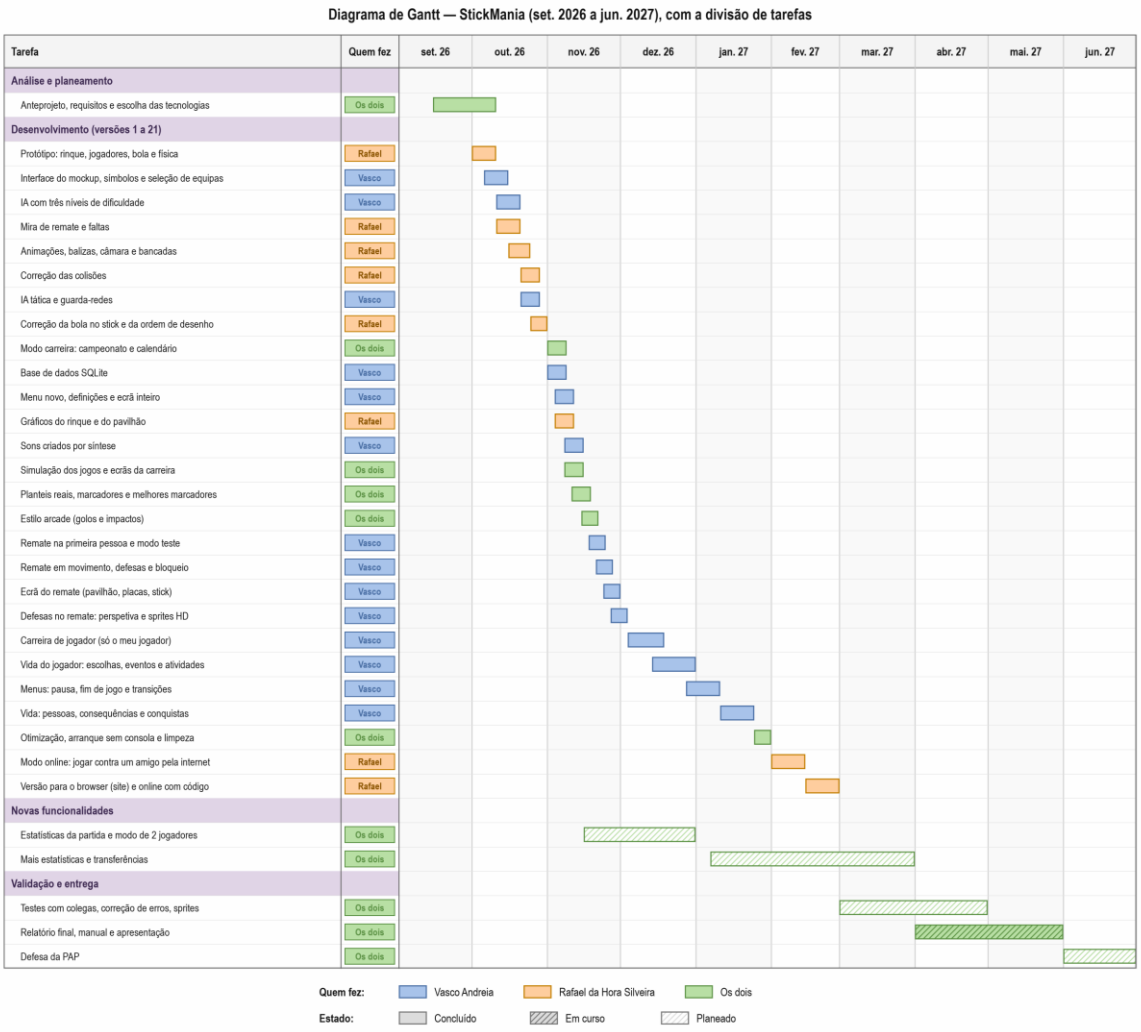


Figura 19 – Diagrama de Gantt do projeto, com a divisão de tarefas (setembro de 2026 a junho de 2027)

O mesmo diagrama de Gantt está também no GitHub, no ficheiro docs/GANTT.md (github.com/RafaelSilveira23/Stickmaniapap/blob/main/docs/GANTT.md). Está escrito em Mermaid, uma linguagem de texto para diagramas que o GitHub desenha automaticamente: cada tarefa é uma linha com o nome (e, entre parênteses, quem a fez), o estado (done para concluída, active para em curso) e as datas de início e de fim. O ficheiro é gerado pelo mesmo script que faz a imagem da Figura 19 (gerar_gantt.py), a partir da mesma lista de tarefas, por isso os dois têm sempre as mesmas datas, e por baixo do diagrama há uma tabela com todas as tarefas.

5.2 Gestão de riscos ◆ MADS

Tabela 10 – Gestão de riscos

Risco	Prob.	Impacto	Medida
Perder código (avaria, ficheiro apagado)	Média	Alto	Tudo no GitHub; commits frequentes
Uma alteração estragar o que funcionava	Alta	Médio	Testar antes de cada commit; reverter com o Git
Atrasos no cronograma	Média	Alto	Versões pequenas e jogáveis; prioridades definidas
Jogo lento em computadores fracos	Baixa	Médio	Pré-desenhar o rinque; medir os FPS
Erros de física (bola fora do rinque)	Média	Alto	Testes automáticos de colisões e golos
Uso dos símbolos dos clubes	Baixa	Médio	Uso apenas educativo, sem fins comerciais
Ausência de um elemento do grupo	Baixa	Médio	Código modular e documentado; repositório partilhado

5.3 Controlo de versões e trabalho em equipa (Git/GitHub)

O projeto usa o Git para o controlo de versões, com o repositório no GitHub (github.com/RafaelSilveira23/Stickmaniapap). Cada conjunto de alterações é guardado num commit com uma mensagem a explicar o que mudou, por exemplo «feat: níveis de dificuldade da IA, redução de quedas e sistema completo de faltas». Assim é possível ver o histórico, comparar versões e voltar atrás, como aconteceu com a folha de sprites que foi revertida.

O código está dividido em módulos (física, IA, interface, câmara, rinque), o que permite a cada elemento do grupo trabalhar numa parte sem estragar as outras. Os diagramas também estão no repositório (pasta docs/diagramas), em formato draw.io, para poderem ser editados.

5.4 Divisão de tarefas

O projeto foi feito em dupla. O trabalho foi dividido por partes do jogo, cada uma com os seus módulos, para cada elemento poder trabalhar sem estragar o trabalho do outro (Tabela 11). A base do jogo, o modo carreira, o estilo arcade, a análise, os diagramas, os testes e o relatório foram feitos pelos dois. As alterações de cada um foram juntadas no repositório do GitHub. O diagrama de Gantt (Figura 19, na secção 5.1) também mostra quem fez cada tarefa e quando.

Tabela 11 – Divisão de tarefas entre os dois elementos do grupo

Parte do projeto	Módulos (ficheiros)	Quem fez
Inteligência artificial e guarda-redes	ai.py, goalkeeper.py	Vasco Andreia
Menus e interface	menu.py, ui.py, componentes.py, graficos.py	Vasco Andreia

Parte do projeto	Módulos (ficheiros)	Quem fez
Sons	sons.py	Vasco Andreia
Base de dados (SQLite)	carreira.py, carreira_jogador.py (parte da base de dados)	Vasco Andreia
Carreira de jogador e vida do jogador	carreira_jogador.py, carreira_jogador_menu.py, vida.py	Vasco Andreia
Remate na primeira pessoa e modo teste	remate_pov.py, pavilhao_pov.py	Vasco Andreia
Física da bola e dos jogadores, regras	**ball.py, player.py, baliza.py**	**Rafael da Hora Silveira**
Gráficos, sprites e câmara em perspetiva	**camera.py, rink.py, animation.py**	**Rafael da Hora Silveira**
Jogo online e versão para o browser	**rede.py, online.py, online_menu.py, rede_web.py, navegador.py**	**Rafael da Hora Silveira**
Ciclo principal do jogo	game.py, settings.py	Os dois
Modo carreira	carreira.py, carreira_menu.py, team.py	Os dois
Estilo arcade	efeitos.py	Os dois
Análise, diagramas, testes e relatório	docs/, ferramentas/	Os dois

Na Tabela 11 estão a negrito as partes feitas por Rafael da Hora Silveira. Neste relatório, essas partes estão descritas em pormenor na secção 3.2.1, as feitas em conjunto na secção 3.2.2, e as feitas por Vasco Andreia estão resumidas na secção 3.2.3.

6. Conclusão

O StickMania cumpre o objetivo principal: é um jogo de hóquei em patins jogável, fluido e com as equipas portuguesas. Tem física da bola com atrito e ressaltos, uma IA com papéis táticos e três níveis de dificuldade, guarda-redes, faltas, duas partes, menus completos, uma apresentação parecida com a televisão, com câmara em perspetiva, bancadas e balizas em 3D, sons criados por síntese e um modo carreira em que se joga (ou simula) um campeonato completo, guardado numa base de dados SQLite. Desde a versão 20 também se pode jogar online, contra um amigo noutro computador, e desde a versão 21 basta abrir o site rafaelsilveira23.github.io/Stickmaniapap no browser, sem instalar nada.

O projeto permitiu aplicar e aprofundar muitas matérias do curso: programação orientada a objetos, organização em módulos, matemática aplicada (vetores, equações do 2.º grau, perspetiva), algoritmos de decisão, testes e controlo de versões. As maiores dificuldades foram a física das colisões e a câmara em perspetiva, resolvidas separando a lógica do jogo da forma como é desenhado.

Contributo individual. No StickMania, o Rafael da Hora Silveira programou a física da bola e dos jogadores (atrito, ressaltos, colisões e desarmes), os gráficos, os sprites e a câmara em perspetiva, e o jogo online, no computador e no browser, com a publicação do site. Nestas partes, as maiores dificuldades foram as colisões (a bola a atravessar as tabelas e os jogadores colados uns aos outros), a câmara em perspetiva sem mudar a física e pôr o jogo a correr e a jogar online no browser. O modo carreira, o estilo arcade, a análise, os testes e o relatório foram feitos em conjunto com o Vasco Andreia.

Como trabalho futuro estão previstos: música nos menus, mais estatísticas por jogador (remates, assistências, defesas do guarda-redes), transferências entre épocas, um modo de dois jogadores no mesmo teclado e sprites novos com mais animações.

7. Anexos

Anexo A – Ficheiros dos diagramas. Todos os diagramas deste relatório estão na pasta docs/diagramas do projeto, em formato .drawio (abrem e editam-se em app.diagrams.net) e em imagem .png:

- 01_casos_de_uso.drawio – diagrama de casos de uso
- 02_arquitetura.drawio – arquitetura em blocos
- 03_estados_jogo.drawio – máquina de estados do jogo
- 04_fluxograma_ia.drawio – fluxograma da IA
- 05_fluxograma_ciclo.drawio – fluxograma do ciclo principal
- 06_classes.drawio – diagrama de classes
- 07_modelo_dados.drawio – modelo de dados SQLite
- 08_fluxograma_carreira.drawio – fluxograma do modo carreira
- 09_gantt.png – diagrama de Gantt (gerado pelo script gerar_gantt.py)
- GANTT.md – o diagrama de Gantt em Mermaid, que o GitHub mostra desenhado (gerado pelo mesmo script)

Anexo B – Manual de utilizador (resumo). 1) Instalar o Python 3.10 ou mais recente. 2) Fazer duplo clique em jogar.vbs (se faltar o pygame-ce, abre o jogar_com_consola.bat, que o instala). 3) No menu, escolher «Jogo rápido» (escolher as duas equipas e a dificuldade) ou «Modo carreira» (escolher a equipa e jogar o campeonato; o progresso fica gravado sozinho; em cada jornada pode jogar ou simular o jogo). 4) Jogar com os controlos da Tabela 3. Para rematar, carregar em X com a bola, apontar com o rato e manter X carregado para escolher a força. O MODO TESTE do menu serve para treinar remates. A engrenagem no canto do menu abre as definições (lista de controlos, ecrã inteiro, estilo arcade e volume do som).

Anexo C – Código-fonte: github.com/RafaelSilveira23/Stickmaniapap